



中國石油大學 (华东)
CHINA UNIVERSITY OF PETROLEUM

本科毕业设计（论文）

题 目：基于 ECS 架构的类吸血鬼幸存者游戏开发及优化（校外）

学生姓名： 刘瀚文

学 号： 2207020509

专业班级： 软件工程 2205 班

指导教师： 董玉坤

2026 年 5 月 26 日

学位论文原创性声明

本人所提交的学位论文《基于 ECS 架构的类吸血鬼幸存者游戏开发及优化（校外）》，是在导师的指导下，独立进行研究工作所取得的原创性成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中标明。

本声明的法律后果由本人承担。

论文作者（签名）：

年 月 日

指导教师确认（签名）：

年 月 日

学位论文版权使用授权书

本学位论文作者完全了解中国石油大学（华东）有权保留并向国家有关部门或机构送交学位论文的复印件和磁盘，允许论文被查阅和借阅。本人授权中国石油大学（华东）可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其它复制手段保存、汇编学位论文。

保密的学位论文在_____年解密后适用本授权书。

论文作者（签名）：

年 月 日

指导教师（签名）：

年 月 日

摘 要

HTML5 与 WebGL 普及后，浏览器端动作游戏要在模块解耦与同屏性能之间取舍；类吸血鬼幸存者玩法又常叠加 Roguelite 单局构筑与元游戏流程，客户端更需在数据驱动、模块划分与可测性上划清边界。针对上述约束，本文以毕业设计项目 Survivor And Fight 为对象，设计并实现基于 LayaAir 3.x 与 TypeScript 的 2D 俯视角生存战斗原型。系统采用五层架构组织代码，Gameplay 以实体组件系统（ECS）承载战斗与技能逻辑，界面层以 MVC 分离视图与状态，数值与技能等内容由 JSON 配表驱动；性能方面通过子弹与怪物对象池复用节点、Web Worker 分担高密度追逐与分离计算，并结合动态图集降低 Draw Call。工程上完成需求指标梳理、总体方案设计、核心模块与主循环实现，并以单元脚本、功能用例与压力实验进行验证。原型可稳定跑通菜单、跑图、战斗与技能装配等闭环流程。

关键词：实体组件系统；类吸血鬼幸存者；系统设计与实现；LayaAir；数据驱动

Abstract

With HTML5 and WebGL widely adopted in browser games, action titles must reconcile modular architecture with runtime performance under large on-screen entity counts. Vampire-survivor-like gameplay further combines Roguelite run-time build choices with meta-game progression, demanding clear boundaries among data-driven content, module separation, and testability. To address these constraints, this thesis presents Survivor And Fight, an off-campus graduation project: a 2D top-down survival combat prototype built with LayaAir 3.x and TypeScript. The system is organized in a five-layer architecture: gameplay logic runs in an Entity - Component - System (ECS) model, the UI follows MVC to decouple views from state, and skills and stats are driven by JSON configuration tables. For performance, bullet and monster object pools reuse scene nodes, a Web Worker offloads chase, separation, and steering when entity counts are high, and dynamic texture atlasing reduces draw calls. The work covers requirement specification, overall design, core modules and the main game loop, and validation through unit scripts, functional test cases, and stress experiments. The prototype delivers a complete loop of menus, run-map navigation, combat, and skill loadout. Experiments show that object pooling and dynamic atlasing effectively mitigate frame-rate jitter under heavy on-screen load.

Keywords: ECS, vampire-survivor-like, system design, LayaAir, data-driven

目 录

第 1 章 序言	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.3 主要创新点与特色	2
1.4 伦理与合规说明	2
1.5 相关游戏与竞品简要分析	3
1.6 术语与缩写预备说明	3
1.7 本章小结	3
第 2 章 相关技术与理论基础	4
2.1 引擎语言选择	4
2.2 项目玩法架构选择	6
2.3 UI 架构选择	8
2.4 本章小结	9
第 3 章 系统需求分析	10
3.1 可行性分析	10
3.2 用户角色与运行环境	10
3.3 功能性需求	12
3.3.1 用例：局内战斗	12
3.3.2 用例：元游戏进入战斗	13
3.4 功能需求详细说明	14
3.5 非功能性需求	15
3.6 数据、接口与约束	15
3.7 需求优先级	16
3.8 需求与测试映射	16
3.9 本章小结	16
第 4 章 系统总体设计	17
4.1 设计原则	17
4.2 系统具体设计	17

4.2.1 基于 ECS 的 Gameplay 实现	18
4.2.2 基于 MVC 的 UI 结构实现	19
4.2.3 基于 Web Worker 和对象池的优化实现	20
4.3 系统交互与时序	21
4.4 可靠性设计	23
4.5 容错与降级策略	24
4.6 可扩展性设计	24
4.7 本章小结	25
第 5 章 核心模块详细设计与实现	26
5.1 ECS 架构 GamePlay 实现	26
5.2 MVC 架构 UI 实现	28
5.3 Web Worker 和对象池的优化实现	29
5.4 游戏主循环设计	31
5.5 典型场景实现剖析	32
5.6 关键代码文件索引	33
5.7 功能模块与论文章节对照	34
5.8 本章小结	34
第 6 章 性能优化与工程实践	35
6.1 性能目标与瓶颈	35
6.2 对象池	35
6.3 Web Worker 卸载	35
6.4 渲染与逻辑侧其它优化	36
6.5 帧预算与内存	37
6.6 性能实验设计	37
6.8 本章小结	38
第 7 章 系统测试与验证	39
7.1 测试策略	39
7.2 单元验证	39
7.3 功能测试	40
7.4 性能测试	41

7.4.1 测试环境	41
7.4.2 测试结果	41
7.5 系统运行截图	42
7.6 测试结论	42
7.7 本章小结	45
第 8 章 总结与展望	46
8.1 工作总结	46
8.2 不足与展望	46
8.3 社会、健康与文化影响	46
8.4 绿色节能与兼容性	47
8.5 本章小结	47
致 谢	48
参考文献	49
附录 A 名词术语及缩略词	51
附录 B 核心配表字段示例（节选）	52
附录 C 需求与测试用例映射（节选）	53
附录 D 缺陷记录（项目实测）	54

第 1 章 序言

1.1 研究背景与意义

移动互联网和 Web 技术普及后，H5 游戏在独立开发和教学里用得越来越多。和安装包相比，H5 可以链接即玩、跨端发布，逻辑和资源也走 Web 栈，改版本和分发更省事。浏览器里 JavaScript 往往只有一条主线程，GC 和绘制共用每帧时间。战斗类若在单帧里堆太多位移、碰撞和界面刷新，帧率会掉，玩家能直接感觉到。

《Vampire Survivors》（吸血鬼幸存者）代表了一类 Survivor-like 玩法：自动攻击、怪潮压力、局内构筑，也常叠 Roguelite 的元进度和随机路线。这类游戏对程序结构有硬要求——数值宜外置、便于改表；同屏怪物和弹幕数量大；菜单、选关、跑图、战斗要串成完整元流程。若仍用深继承的面向对象写法，实体一多，模块容易缠在一起，维护变难，帧率也会吃紧^[1]。

实体组件系统（Entity-Component-System, ECS）把实体、组件数据和系统逻辑分开，用组合代替继承，边界更清楚，也便于按组件类型批量更新。LayaAir 等国产 H5 引擎已覆盖 2D、TypeScript 和多端发布，适合做“引擎 + 玩法架构 + 性能”一体的毕设平台。

本文以浏览器端 2D 俯视角 Roguelite 原型 Survivor And Fight 为对象，技术栈为 LayaAir 3.x 与 TypeScript。工作包括需求分析、总体设计、实现和测试，重点验证轻量 ECS、配表技能、对象池和 Web Worker 在 H5 生存战斗里能否落地。

1.2 国内外研究现状

Gregory 在专著里按工业案例划分渲染、资源、场景图、动画和脚本等子系统，是读商业引擎结构的常用入口^[2]。Munro 等拆解 Quake 系列，便于区分引擎层与游戏层^[3]。Agrahari 等分析 Unreal 的模块组织，反映大型引擎常见的功能切分^[4]。Goslin 等介绍 Panda3D，属于脚本叠在 C++ 核心上的早期做法^[5]。

近几年更关注“改架构会影响哪些模块”。Ullmann 等的 SyDRA 能从开源引擎恢复子系统依赖图；对照实验表明，结构化图有助于判断耦合^[6]。同一团队后续把耦合模式画出来，认为目录嵌套过深、模块绑太紧会导致架构退化^[7]。

H5 还要单独算主线程时间、异步加载和浏览器沙箱。Unity 手册对脚本生命周期、

渲染管线和平台差异有说明，可用来对照浏览器端的限制^[8]。市面教程多数只教功能演示，很少给出“可扩展玩法框架 + 可复现实验”的完整范例。

ECS 用实体 ID、纯数据组件和无状态系统描述对象，按组件类型批量更新，减轻逻辑缠绕，也有利于缓存。Unity DOTS 走数据导向和并行。毕设规模下，用 Map 存组件、单线程 tick 的轻量 ECS 更好上手、也好查错。

在玩法上，Survivor-like 靠波次和自动攻击；Roguelite 常用 DAG 地图选路；也有作品用模块化技能加深构筑。商业产品体量更大，H5 原型若仍用预制体加 OO 继承，实体一多，维护和性能都会吃紧。Briand 等的对照实验说明，OO 结构会直接影响后期改代码的成本^[9]，从侧面支持用 ECS 这类组合式写法。

性能方面，生存战斗常要处理大量单位同屏运动。Reynolds 的 Boids 用局部规则驱动群体，是经典起点^[10]。Thalmann 等总结人群仿真里“个体规则 → 群体现象”的常见做法^[11]。Cantrell 等在 Unity 做物理疏散模型，并走完 V&V 流程^[12]。Bott 等在 UDK 上用物理工具链做人群^[13]。Helbing 的恐慌模型说明，密集场景里分离力很关键^[14]。McKenzie 等把人群模型接到游戏仿真里，讨论实时开销控制^[15]。Web Worker 不能碰 DOM 和多数引擎 API，碰撞和渲染应留主线程；Worker 只适合搬纯数值计算。

1.3 主要创新点与特色

本项目工程侧特点如下：

(1) 轻量 ECS：EcsWorld 驱动玩法 tick，ViewComponent 只绑 Laya 节点，业务尽量不直接调引擎 API。

(2) 性能措施：对象池来减少频繁实例化；引入 WebWorker 进行多线程运算，降低主线程压力；采用图集降低渲染压力。

(3) UI 与战斗协同：UIStackManager 管理全屏页面栈；打开 Tab 装配面板时 GameSession.paused 暂停战斗逻辑，SkillLoadoutSyncSystem 把栏位写回施法状态。

1.4 伦理与合规说明

内容为虚构战斗，无真实暴力。不采集个人敏感信息。若对外发布，需遵守网游与未成年人相关规定，并提示适龄与时长^[18]。社会影响在第 8 章另有说明。

1.5 相关游戏与竞品简要分析

吸血鬼幸存者：极简操作加自动攻击和怪海，证明品类成立^[7]。本项目借其节奏和升级感，但用 ECS 和 JSON 表把逻辑摊开，方便写进论文。

杀戮尖塔：节点地图加路线风险^[7]。本文跑图也用 DAG，战斗却是实时俯视，瓶颈在帧率和碰撞。

Noita：模块拼法术带来深度^[8]。本文 effect 链只吸收“组合”思路，不做像素物理。

其他 H5 生存作：不少仍用预制体加 OOP。本文用 ECS、配表和 Worker，强调可维护和可测，不拼内容量。

1.6 术语与缩写预备说明

英文缩写首次出现给中文释义，见附录 A。类名与 API 保留英文，便于对照源码。

1.7 本章小结

本章交代背景、相关研究和本项目的工程特点，供后文需求与设计引用。

第2章 相关技术与理论基础

本章从引擎与语言、玩法与逻辑架构、界面架构三条线说明 *Survivor And Fight* 的技术选型及依据，为第3章需求与第4章设计提供共同语境。各节在文献与业界实践基础上给出毕业设计规模下的取舍，并与后文 *SimpleEcsDemo*、*UIStackManager* 等模块对应，避免孤立罗列概念。

2.1 引擎语言选择

浏览器 2D 游戏通常二选一：Phaser、PixiJS 这类渲染库，或 Cocos、Laya 这类带 IDE 的引擎。库轻，但场景、预制体、UI 和打包往往要自己拼；引擎把帧循环和输入包好了，更适合课设周期内做出能点的 Demo^[2]。

本项目采用 LayaAir 3.x。实习期间参与过商业 H5 项目，客户端用 Laya，玩法逻辑用 ECS 拆分，与本文原型路线一致，也降低了从 Demo 到可维护工程的风险。Laya 3.x 原生 TypeScript，IDE 内可直接做 2D 预制体、Area2D 与 UI2（GBox、GButton 等），元菜单、跑图和战斗 HUD 不必另拼工具链。Gregory 指出，商业引擎把渲染、资源、场景更新与脚本生命周期统一纳入运行时管理；在 H5 场景下，Laya Air 承担的是同类平台角色，而非仅作渲染库。

同题材若用 Unity、Unreal 做为游戏引擎，帧预算多花在原生渲染与物理上；浏览器端则受单线程和 GC 牵制，同屏实体一多，性能弱点更明显。本文因此把 Laya 节点限于表现绑定，核心 tick 放在自研 ECS，与实习项目里的分层做法一致。Ullmann 等对引擎子系统耦合的研究表明，无论引擎种类，划分“引擎层—游戏逻辑层”都有利于维护^[7]。因此 Laya 节点仅作表现层，核心玩法放在自研轻量 ECS 中更新，而非全部写入 Laya.Script 回调。

表 2-1 主流 H5/跨平台方案与本项目选型对比

方案	优势	劣势	本项目
Phaser/PixiJS	轻量、社区活跃	工作流需自建	未采用
Cocos Creator	生态成熟、工具链完整	与现有课设栈不一致	未采用
LayaAir 3.x	IDE 一体、TS 原生、2D/UI2 完备	文档与社区小于头部引擎	采用

客户端脚本定为 TypeScript。它在 JavaScript 运行时之上做静态类型检查，并用模块把代码切块；实体、组件、系统继续增加时，类型约束能稳住对外接口。Briand 等就面向对象设计可维护性做过对照实验：模块边界划清、命名风格统一，后续阅读与修改会省一些工夫^[4]。战斗可调常量、UI 阈值、路径正则等很少改、却必须写进代码的量，收进 `src/defines/`，由 `index.ts` 统一 re-export，各业务文件不在开头堆魔法数。会频繁迭代的策划数值放进外置 JSON 配表（见第 2.2 节）。前者叫工程常量，后者叫策划参数，两类分开管。

主循环由 Laya.timer.frameLoop 驱动：每帧先 EcsWorld.update(deltaTime)，再由引擎渲染并分发 UI 事件，符合 Gregory 所述逻辑 tick 与绘制分离的常见做法。入口 Main.ts 挂在 Area2D 子节点而非 3D 根节点，避免纯 2D Demo 进入 3D 渲染管线。

使用 Laya API 须统一 Laya. 前缀（如 Laya.Sprite、Laya.Handler.create）。角色与子弹经 Laya.Prefab.instantiate 生成。可加载资源（材质、贴图、预制体）须放在 assets/，不可放在 src/，否则预览路径解析失败。2D 位移须用 transform.position = new Laya.Vector3() 或 translate，不可对 position getter 返回的副本调用 setValue，否则逻辑坐标已变而画面不动——该问题在相机跟随与怪物追逐调试中多次出现，本文记录以供规避。上述主循环调用关系如图 2-1 所示。

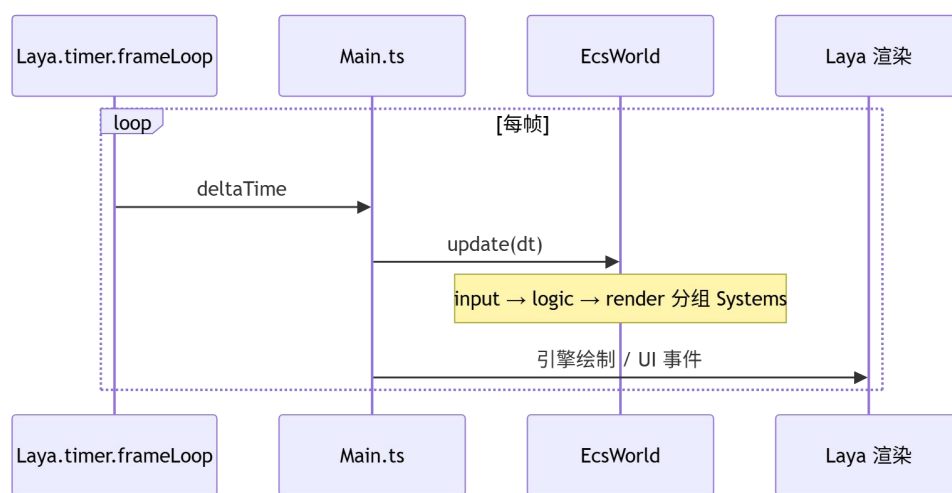


图 2-1 主循环流程图

2.2 项目玩法架构选择

一局里玩家主要在躲、自动打、捡经验、升技能，怪会越刷越密。射击技巧让位给走位和构筑。程序上要扛住三件事：移动时碰撞和结算不能飘；几分钟内多次升级，奖励得立刻写进属性或栏位；调数值尽量改表，别动战斗内核。

SimpleEcsDemo 默认创建玩家并由 PlayerAutoCastSystem 自动施法，ControlSystem 处理走位，以降低操作门槛、突出构筑。经验由 ExperienceSystem 处理；击杀奖励与等级加成通过 defines 中 MONSTER_EXP_REWARD_BASE、MONSTER_EXP_REWARD_LEVEL_BONUS 等常量调节，属于可配置节奏而非硬编码。

实体组件系统（ECS）将对象拆为实体标识（Entity）、纯数据组件（Component）与无状态系统（System）。新增怪物行为时往往只需增 System 或扩展组件字段，无需维护大继承树；按类型批量处理更方便。

Unity DOTS 采用 ECS + Job System + Burst，面向数万实体并行^[7]。毕业设计周期与团队规模有限，完整 DOTS 等价物学习与 Laya 节点绑定成本过高。本项目采用教学向轻量 ECS：EntityId 为数值；ComponentStore 按组件类型用 Map 存储；EcsWorld.update 单线程顺序调用已注册系统，按 input/logic/physics/render 分组排序。第 1 期目标是结构清晰、支撑数百同屏实体，不追求极致数据导向布局。

表 2-2 Unity DOTS 与项目轻量 ECS 对比

维度	Unity DOTS	本项目 ECS
并行模型	Job System 多线程	单线程；追逐等片段可选 Web Worker
内存布局与引擎绑定	Archetype/Entities Graphics	按组件类型的 MapViewComponent 绑定 Laya 节点
适用规模	数万实体级	数百实体级
迭代与调试成本	高	中低

玩法层按 ECS 拆成组件与系统两块。组件一侧：PlayerTag、MonsterTag 标记实体类型；Position、Velocity 管坐标与速度；Attribute 存基础数值，modifier 带来源字段，结算时可追溯；另有 Skill、SkillLoadoutState 管技能与栏位。系统一侧：MovementSystem 推进位移，AttributeSystem 重算有效属性，SkillSystem、BulletSystem 处理施法与子弹，MonsterChaseSystem 负责怪物追击。以上名单为文中举例，并非封闭列

表。

FilterRegistry 事先注册 Players、Monsters 等筛选名。各 System 按名取实体，不必各自拼组件交集条件。

玩家实体与怪物实体挂同一套位移、属性、技能组件，差别只在 PlayerTag 或 MonsterTag。新行为靠增删组件或系统扩展，不拉 MonsterBase 一类继承链，符合 ECS 里组合优于继承的惯例。

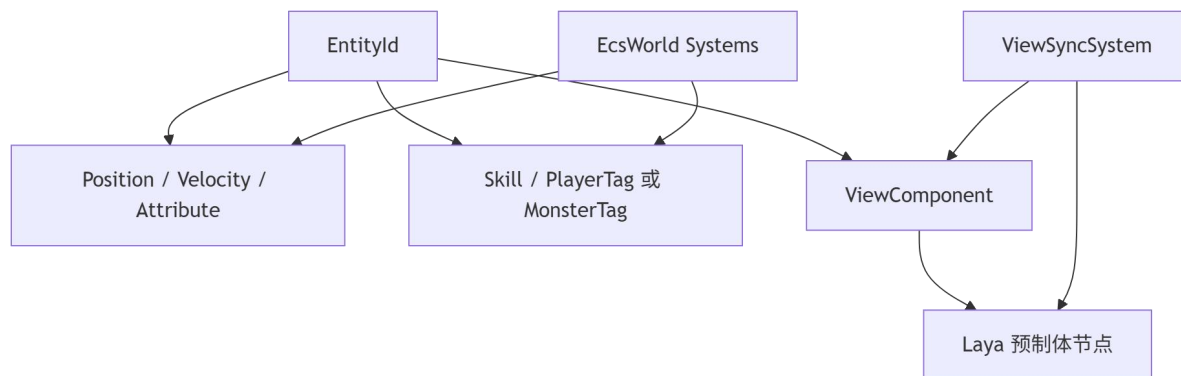


图 2-2 ECS 绑定关系图

Roguelite 与动作游戏普遍数据驱动：冷却、伤害、子弹参数、效果链外置为表，运行时加载^[7]。常见做法包括：（1）运行时加载 JSON 并反射填充数据类；（2）构建期生成 TypeScript 常量；（3）注册表声明表结构、运行时按路径加载。本项目用第（3）种：

各张表的表名与 JSON 路径写在 tables.registry.json。启动阶段调用 ConfigBootstrap.ensureGameConfigLoaded() 拉取数据；initData 再把每一行挂进 Data.Character、Data.Skill 等命名空间，后续逻辑按命名空间读表即可。

一条技能在 Skill 表占一行，具体效果拆到多行 SkillEffect。单行可配置子弹、穿透/分裂/连锁类修饰，或直接伤害。EffectExecutor 按表内顺序自上而下解释；若某颗子弹要带修饰，对应修饰行必须写在它的 bullet 行之前。伤害多写成 "atk*1.5+10" 这类表达式字符串，由 FormulaParser 在白名单运算符内求值，工程里不用 eval。索敌与指向交给 Targeting，支持自动选目标和简易手动指向。

JSON 方便版本管理和 diff，但错配往往集成时才暴露。启动用 isGameConfigReady 拦一道，公式和修饰器另有 *.verify.ts 脚本。

元游戏参考 Slay the Spire，用 DAG 串起一局流程^[7]。一局分三个 Act；每个 A

ct 是一张多层有向无环图，节点类型包括休息、普通战、Boss、宝藏等。常见生成步骤有：按层摆节点、按权重抽类型、限制连边、再做过可达性检查。本项目由 RunMapGenerator 生成固定行数网格——Boss 固定在最后一行，中间行随机分叉；validateActReachBoss 用来判断 Boss 是否可达。随机多次仍失败时，退回预设保底图，避免玩家卡在到不了 Boss 的图上。

SeededRng 固定种子后，同一种子会生成同一张地图，答辩演示和回归测试都能复现。玩家确认节点后，MetaFlowController 调用 onEnterCombat，把节点参数（例如 Boss 关延长生存时间）传给战斗场景。元游戏流程与局内 ECS 分开维护，两边只通过约定接口交换数据。

Survivor-like 后期同屏怪物与弹幕增多，H5 热点包括：每帧大量碰撞、预制体反复实例化触发 GC、主线程追逐与分离计算。常见对策有对象池、纹理合批、逻辑暂停、纯数值计算迁至 Web Worker^[15]。

BulletPool、MonsterPool 按预制体路径分桶，复用 Laya 节点，减少反复 instantiate/destroy。BulletHitTest 用距离平方比较远近，不必每帧开方。技能面板打开时，GameSession.paused 为真，战斗类 System 在 update 开头直接返回。场上实体数超过阈值后，CombatDataPrepareSystem 打包坐标等快照，经 CombatDataBridge 发给 Worker；追逐、分离、摆动在共用的 combatWorkerLogic.ts 里计算，主线程只把速度写回组件。子弹碰撞仍依赖 Laya 节点和阵营标记，这部分留在主线程处理。Cantrell 等在 Unity 做物理人群疏散时同样强调重计算与交互渲染分离^[12]；Helbing 等关于恐慌人群的研究提示密集场景须关注分离力与稳定性，与 MonsterChaseSystem 中分离项设计一致。

以上几项选型在「方便扩展」和「方便测试」之间折中：ECS 加 JSON 配表负责内容和数值迭代；对象池与 Worker 顶住同屏实体规模；DAG 跑图管住元游戏节奏。三者各管一块，没有互相替代。

2.3 UI 架构选择

游戏 UI 常见模式有 MVC、MVP、MVVM 等^[8]。Heijstek 等比较架构描述的图文形式，指出结构化图示有助于理解系统边界；本项目采用 MVC 变体：

Model：hp、装配状态等来自 ECS，跑图另有 RunMapPanelModel、RunMapState；

View：预制体和 UI2 节点只管摆位和显示；

Controller: `UiControllerBase` 子类管生命周期和路由，不碰战斗场景里的实体节点。

开始、选关、跑图都是全屏页。若多个面板一起 `visible`，容易挡操作、事件乱窜。`UIStackManager` 用栈管路由：`push` 压入并显示，`pop` 回到上一层，同一时刻只有一个全屏页拿焦点。

局内界面包括主 HUD（血条、经验）、升级奖励、死亡重启，以及 Tab 打开的技能装配面板。`SkillSelectPanelController` 监听 Tab：面板显示时 `GameSession.paused` 为真，战斗 `System` 在 `update` 入口 `return`，怪物和子弹逻辑停住；指针事件仍由 UI 接收。战斗 `tick` 与界面输入分开，改装配时不会继续挨打。

技能栏位用长按拖拽调整。`SkillDragService` 在按下超过阈值后，在顶层挂一张代理图标，避免被 `GBox` 裁剪后点不中。`SkillSlotHitTest` 在同一套坐标系里判断落点是装备栏还是效果槽。关面板时，`SkillLoadoutSyncSystem` 一次性把 `SkillLoadoutState` 写回 `Skill` 和 `PlayerAutoCastSystem`，防止栏位只改了一半却还在放旧技能。图标路径来自配表 `iconPath`，由 `SkillIconBinder` 加载；资源文件名与技能 `id` 按目录约定对齐。

跑图页 `RunMapPanelView` 根据 DAG 状态画节点和连线；`MapNodeIconUtil`、`LineLayoutUtil` 管图标与折线布局。玩家只能沿当前节点相邻、且已解锁的边前进；能不能点由 `Model` 校验，`View` 里不写通行规则。图界面 `RunMapPanelView` 按 DAG 状态渲染节点与连线，`MapNodeIconUtil`、`LineLayoutUtil` 负责布局；玩家只能选与当前节点相邻且已解锁的边，规则由模型层校验，不在 `View` 硬编码。

2.4 本章小结

本章从引擎语言、玩法架构、UI 架构三方面说明 LayaAir + TypeScript、轻量 ECS + 配表 + Roguelite 跑图、MVC + UI 栈的技术路线及取舍，将在第 4 章落实为 ECS/MVC/Worker 设计，在第 5–7 章经实现与测试验证^[17]。

表 2-3 UI 架构选型要点汇总

问题	选型	理由
全屏导航	<code>UIStackManager</code> 路由栈	避免多面板同显
局内构筑	Tab+拖拽+ <code>SyncSystem</code>	暂停战斗、原子同步
数据绑定	预制体 <code>View+Controller</code>	便于美术替换与动画
与战斗边界	禁止 <code>System</code> 操作 UI 节点	单向依赖、利于测试

第3章 系统需求分析

本章在第2章技术选型基础上，给出 Survivor And Fight 的可行性论证、功能与非功能需求及验收口径，为第4章总体设计与第7章测试提供依据。需求描述以可验证为原则：每条 Must 级需求均可在代码或测试用例中找到对应实现或断言。

3.1 可行性分析

代从现有工程来看，本课题已经搭起 LayaAir 3.x 加 TypeScript 的 H5 客户端，局内玩法走轻量 ECS，数值和技能走 JSON 配表，菜单、DAG 跑图、Tab 技能装配、对象池和 Web Worker 也都有对应模块，不是停在纸面方案上。Laya 提供 2D 预制体加载和 frameLoop 帧回调，答辩用的 IDE 内置 Chromium 同时支持 WebGL2 和 Worker，和文献里讨论的引擎子系统边界、在商业引擎上叠领域逻辑的做法也能对上^[6]。

配表是否就绪由 ConfigBootstrap 里的 isGameConfigReady()判断，它要求 configLoaded 为真且 Data.Skill.GetAll()的长度大于 0；ensureGameConfigLoaded()若发现已经就绪就直接返回 true，否则把 loadAllTables()包进单例 loadPromise，保证全局只加载一遍。这条链路与图 4-2 最左侧「应用启动」一致，双通道读表细节见图 5-4。loadAllTables()遍历 configBases()读 tables.registry.json 再 initData 各表，skillCount 大于 0 时置 configLoaded=true。ECS、FormulaParser、属性修饰器配有 verify 脚本，改底层可不进场景回归。风险在千怪帧率、Worker 与主线程是否共用 combatWorkerLogic、UI2 拖拽命中，第7章五波实验和 T-F-05/06 已部分覆盖，课设周期内交付原型和论文可行。

本课题按教学原型交付，成本主要在开发与测试，不涉及服务器运维和支付接口。题材为奇幻战斗，正文与演示须克制暴力表现，并提示控制游戏时长；跑图里的休息节点用来调节节奏。程序不采集用户敏感信息，定位是单机演示，不做账号与联网运营。

3.2 用户角色与运行环境

系统仅设玩家单一角色：浏览主菜单、选关或跑图、局内移动与构筑、死亡后重启。无管理员后台；未来若扩展存档，仍保持“玩家”为唯一角色。

表 3-1 运行环境建议配置

类别	要求
处理器	四核 2.0 GHz 及以上
内存	8 GB 及以上
显卡	支持 WebGL2
浏览器	Chrome / Edge 最新稳定版（答辩演示以 IDE 内置 Chromium 为准）
开发环境	LayaAir IDE 3.x、Node.js（工具脚本）、TypeScript

本课题交付的是浏览器端单机 H5 原型，运行环境指玩家或答辩演示时实际打开游戏所用的软硬件条件；开发环境指作者编写、导出配表与跑单元脚本时的工具链。二者有交集但不等同：表 3-1 同时列出两类要求，本节对表中条目作说明，并补充表中未单独成列、却影响能否正常进战斗的约束。

（1）硬件与显示。客户端逻辑与绘制均在用户本机完成，不依赖专用服务器。处理器、内存与显卡需满足表 3-1 下限，以保证 WebGL2 上下文能创建、千怪量级实验时主线程仍有时间预算。演示与第 7 章性能数据统一在 1920×1080 分辨率下采集，答辩以 LayaAir IDE 内置 Chromium 预览为准；若改用 Chrome/Edge 独立窗口，须仍为支持 WebGL2 的 Chromium 内核，否则 Worker、合批与帧率数据不可与论文表格直接对比。

（2）浏览器运行时能力。除表 3-1 所列浏览器外，运行时尚须：JavaScript 已启用；支持 WebGL2（2D 预制体与战斗画面）；支持 Web Worker（用于追逐重算卸载，失败时须能回退到主线程 computeSync，见第 4.5 节）；能通过 fetch 或引擎 Laya.loader 读取本地或同源 JSON 配表（tables.registry.json 及各表文件），且响应为合法 JSON 而非 404 页面。游戏为单机本地运行，不要求公网、账号服务或数据库；局内状态保存在内存与 ECS 组件中，刷新页面即重新开始。

（3）开发与构建环境（表 3-1「开发环境」行）。源码在 LayaAir IDE 3.x 中组织与预览，语言为 TypeScript，与引擎脚本生命周期对接。

（4）资源与目录约束。发布或预览前须将配表置于运行时可访问路径（工程内通过 ConfigBootstrap 的 configBases() 解析；开发阶段常把 docs/config 同步到预览目录下的 config/）。未执行配表导出时 isGameConfigReady() 可能长期为假，Main 无法进入战斗，故运行环境在软件层面包含「配表文件随包可达」这一前提。预制体、UI 预制与图集资源由 Laya 资源管线加载，不单独要求 CDN，但磁盘需能容纳工程资源

体积。

(5) 与非功能需求的关系。运行环境达标是达到第 3.5 节指标的前提：常规波次均值帧率、千怪 P95、Worker 与主线程轨迹一致等，均在表 3-1 所述浏览器与分辨率下测量（见表 3-3、第 7 章）。不包含的内容：多人在线、移动端适配、无 WebGL 的纯 Canvas 降级、生产环境运维监控等，均超出本课设范围。

3.3 功能性需求

表 3-2 汇总主要功能模块；详细条目见第 3.4 节。

表 3-2 主要功能性需求一览

模块	需求描述
ECS 核心玩法实体	创建/销毁实体；按类型存取组件；分组更新 玩家/怪物标签；移动、视图同步；hp/maxHp 与 modifier
技能战斗	多 effect 技能；自动施法；子弹表驱动伤害与穿透/分裂/连锁
怪物系统	波次刷怪；追逐与分离；接触伤害；离屏回收
进度系统	经验、升级、随机奖励
元游戏跑图	开始/选关；onEnterCombat；生存计时胜利 三幕 DAG；节点类型；种子可复现；Boss 可达
技能装配 UI	Tab 暂停；三装备栏；长按拖拽装配
配置	JSON 注册加载；缺失时 DEFAULT_*与去重日志

3.3.1 用例：局内战斗

参与者：玩家。

前置条件：isGameConfigReady()==true（配表已加载且 Data.Skill 非空）；SimpleEcsDemo.init()已完成（玩家实体、会话实体 GameSession 及主要 System 已注册）；战斗 Area2D 容器可见。

主成功场景：

(1) 玩家通过 ControlInputAdapter 输入移动，ControlSystem 写入 Velocity，MovementSystem 更新 Position，相机由 Main 跟随，对应图 4-2「战斗帧循环」中每帧 tick。

(2) PlayerAutoCastSystem 按三装备栏检查 Skill.cooldownRemain，通过后 buildSkillCastPlan 按配表顺序执行 effect 链，EffectExecutor 生成子弹或直伤，如图 5-1 所示。

(3) BulletSystem 对子弹做圆碰撞，按配表应用穿透、分裂与连锁；怪物 Attribut

e.hp 减少，击杀触发 ExperienceSystem 加经验，达阈值时 UpgradeRewardSystem 弹出奖励并写回属性或技能。

(4) 玩家按 Tab，SkillSelectPanelController 将 GameSession.paused 置真，战斗 System 在 update 入口 return，UI 仍可响应；玩家长按拖拽改 SkillLoadoutState，关面板后 SkillLoadoutSyncSystem 写回 Skill 与 PlayerAutoCastSystem，paused 恢复 false，新 effect 链在下一冷却周期生效，如图 5-2 所示。

(5) 生存计时未结束前可重复 (2) (3)；界面血条由 BloodBarSyncSystem 与 ECS 同步，见图 7-3、图 7-4。

扩展场景：玩家 $hp \leq 0$ 时，PlayerDeathSystem 置 paused 与 combatFailed，RestartPanelSystem 在 $hp \leq 0$ 且暂停时弹出重启面板（非 Tab 装配误触），见图 7-6；玩家确认后走图 4-2「死亡与重启」清场流程。

验收关联：手工用例 T-F-01、T-F-04（直伤与公式）、T-F-05/06（Tab 装配）；第 5.5.1 节场景一（首次进战斗）、5.5.2 节场景二（Tab 换装配）。

3.3.2 用例：元游戏进入战斗

参与者：玩家。

前置条件：ensureGameConfigLoaded()已成功；元菜单路由已注册（MetaMenuBootstrap）；玩家在开始界面选择「跑图」或「选关」之一。

主成功场景：

(1) 选关路径：玩家在 SelectLevelPanel 确认关卡，MetaFlowController 组装 onEnterCombat 载荷（可含关卡参数），触发回调。

(2) 跑图路径：玩家在 RunMapPanel 沿三幕 DAG 已解锁边前进，节点类型由 RunMapGenerator 与 RunMapState 决定；确认战斗类或 Boss 类节点时，Model 判定合法后同样触发 onEnterCombat，DAG 结构与保底生成如图 5-3 所示，界面见图 7-2。

(3) Main 显示战斗 Area2D、await SimpleEcsDemo.init()，注册局内 System 并生成玩家；CombatSurvivalTimer 开始生存计时，对应图 4-2「进入战斗」段与第 5.5.1 节场景一。

(4) 若节点为 Boss 或配置更长胜利时长，payload 可把胜利所需时间传入战斗逻辑，计时达标后走胜利奖励流程（与局内升级奖励区分）。

扩展场景：RunMapGenerator.generate 多次失败且 validateActReachBoss 不通过时，

系统应走 `generateFallback` 线性保底图，避免局外进度卡死（见第 4.5 节、图 4-7 跑图分支）。

验收关联：T-F-09（菜单进战斗）；第 5.5.1 节；跑图截图图 7-2、主菜单图 7-1。

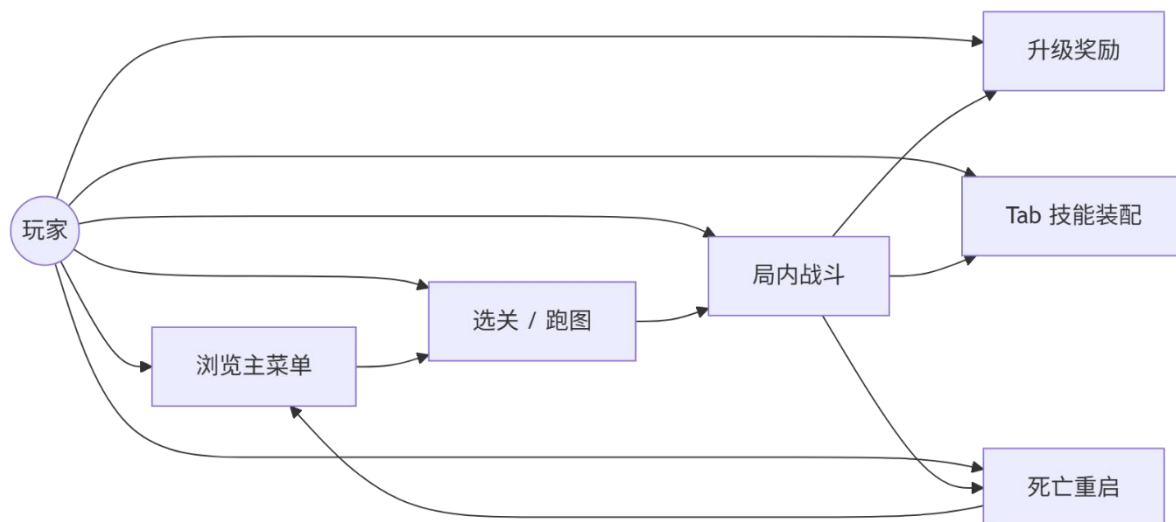


图 3-1 游戏流程图

3.4 功能需求详细说明

ECS 核心在需求上要求实体从创建到销毁整条链路是闭合的，工程里 `EcsWorld` 的 `createEntity()` 会转调 `EntityManager` 发号，`destroyEntity(id)` 则先执行 `components.removeAllComponents(id)`，再 `entities.destroyEntity(id)`，这样组件不会挂在已经销毁的实体上。同类型组件不能在同一实体重复挂载，否则 Buff 和升级奖励的来源会对不上。`System` 通过 `registerSystem(system, group, priority)` 登记，`group` 默认取 `system.group` 或 `logic`，`priority` 默认 0，登记后会 `sortSystems()`，按 `input`、`logic`、`physics`、`render` 的组序排，同组里再按 `priority` 从高到低调 `update(deltaTime)`，主循环因此只要调 `EcsWorld` 这一处入口。查询上既要 `getAllOfType`、`getEntitiesWith`，也要 `FilterRegistry` 在 `registerNamedFilters` 之后提供的 `Players`、`Monsters` 等命名集合。上述约定在 `ecsCorePhase1.verify.ts` 里有脚本覆盖，对应表 3-5“实体销毁清理组件”的验收项。

战斗与技能需求围绕玩家实体和配表驱动展开，玩家侧必须有 `hp`、`maxHp`，`Attribute` 上的 `modifier` 要带来源，升级和 Buff 结算时能追到是哪条 `effect` 写进去的。技能冷却和 `effect` 列表由 `Skill`、`SkillEffect` 表配置，`PlayerAutoCastSystem` 给至少三个装备栏各维护一套 `Skill.cooldownRemain` 和 `pendingCasts`。效果链在实现上与第 5.1.3 节一致，

如图 5-1 所示：buildSkillCastPlan 维护 pendingSplit、pendingChain、pendingPierce，modifier 行只改 pending，bullet 行才写入 BulletSpawnSpec 并 spawnBulletWithOptions，直伤行经公式树求值（图 5-5），表里 modifier 行须排在对应 bullet 行前面。子弹带阵营且连锁半径有上限^[9]。

怪物与元游戏、UI 在流程上要能串起来，MonsterWaveSpawnSystem 在玩家周围环带刷怪并控制间距，MonsterChaseSystem 做追逐和分离，MonsterContactDamageSystem 处理贴脸伤害，MonsterRecycleSystem 配合 MonsterPool 把离屏单位收回去，击杀后 ExperienceSystem 按等级加成发经验。跑图每局生成三幕 DAG，结构示意图如图 5-3 所示，玩家只能沿当前节点已解锁的邻接边前进，RunMapGenerator 多次随机仍过不了 validateActReachBoss 时会走 generateFallback。全屏页走 UIStackManager 的 push、pop，局内 Tab 装配时序如图 5-2 所示，SkillSelectPanelController 把 GameSession.paused 置真，战斗 System 在 update 入口 return，SkillDragService 和 SkillSlotHitTest 区分装备栏与效果槽，BloodBarSyncSystem 把 ECS 数值同步到 ProgressBar^[17]。

3.5 非功能性需求

性能：目标稳定 60 FPS；同屏规模由 MONSTER_COUNT 与波次控制；对象池与 Worker 按第 4.2.3 节启用。

可维护性：模块边界与第 4 章一致；静态常量集中于 defines。

可靠性：Worker/配表/跑图失败时的回退路径见第 4.4 节。

兼容性：优先 Chromium；Worker 不可用时 computeSync 主线程回退^[5]。

表 3-3 非功能需求量化指标（验收参考）

指标	目标值	测量方法
战斗帧率（常规波次）	均值 ≥ 55 FPS	TestLevelFpsTracker
千怪消融 P95	池化后显著低于无池	表 7-6
配表就绪	Data.Skill 非空	isGameConfigReady
Worker 行为	与主线程无逻辑漂移	轨迹对比（见第 7 章）

3.6 数据、接口与约束

数据层面，运行时状态放在 ECS 组件里，角色和技能参数放在 Character、Skill、SkillEffect、Bullet 等配表行里，跑图进度在 RunMapState 的图结构上，局内暂停和装

配在 `GameSession`、`SkillLoadoutState` 上。接口层面，`ensureGameConfigLoaded()` 负责读表并返回是否就绪，`MetaFlowController.onEnterCombat(payload)` 把跑图或选关节点载荷带进战斗，`ControlInputSource` 把输入交给 `ControlSystem`，`UIStackManager.push/pop` 管全屏切换，`CombatWorkerComputeRequest/Response` 字段要保持版本兼容以免 `Worker` 解包失败。约束包括浏览器开启 `JavaScript` 和 `WebGL`、开发阶段执行 `sync-config-to-bin.ps1`、单机本地运行、2D 俯视、同屏按数百实体级设计。

3.7 需求优先级

表 3-4 需求优先级（MoSCoW）

级别	需求
Must	ECS 战斗、子弹碰撞、移动、刷怪、配表、主菜单进战斗、生存胜利
Should	跑图 DAG、技能装配、升级奖励、对象池与 <code>Worker</code>
Could	法杖三槽完整 UI、全量 <code>Effect</code> 独立特效
Won't	联机、账号、支付（本期）

3.8 需求与测试映射

表 3-5 列出需求与验证手段的对应关系，供第 7 章追溯^[18]。

表 3-5 需求与测试映射（节选）

需求	验证方式	编号/脚本
ECS 实体销毁清理组件	单元验证	<code>ecsCorePhase1.verify.ts</code>
Tab 暂停与装配同步	手工用例	T-F-05 、 T-F-06
Boss 可达	生成断言	<code>validateActReachBoss</code>
公式安全解析	单元验证	<code>formulaParser.verify.ts</code>
对象池降 P95	性能消融	表 7-6 第 4 - 5 波

3.9 本章小结

本章写完可行性和需求清单，并挂上 `MoSCoW` 与测试映射。下一章展开分层设计和 `Worker`、可靠性细节。

第4章 系统总体设计

4.1 设计原则

系统总体设计按下列原则组织，后文各模块实现与之对应：

- (1) 数据与逻辑分离：组件只存字段，逻辑写在 `System`；常改的战斗参数放进 `JSON` 配表，很少改的工程常量收在 `src/defines`^[7]。
- (2) 单一职责：`BulletSystem` 管子弹移动和碰撞；`EffectExecutor` 管配表效果解释；`MetaFlowController` 只调度元游戏流程，不直接改 ECS 组件。
- (3) 可验证：公式解析、ECS 核心、属性修饰器等模块配有 `.verify.ts` 脚本；改代码后跑一遍即可做快速回归^[18]。
- (4) 渐进复杂度：先实现轻量 ECS 和单线程 `tick`；对象池和 `Worker` 等怪物数量上来、性能吃紧后再接入。
- (5) 低耦合：UI 通过 `Controller`、`SyncSystem` 读写 ECS，不绕过 `System` 直接改战斗实体。
- (6) 可降级：表没加载、`Worker` 起不来、地图生成失败时，都有回退（第 4.4 节、第 4.5 节）。

4.2 系统具体设计

逻系统按表现层、UI 控制层、游戏逻辑层、领域服务层、基础设施层五层组织，依赖方向自左向右、上层只依赖下层，整体关系如图 4-1 所示。`SimpleEcsDemo` 落在游戏逻辑层，集中注册 ECS `System` 并注入 `BulletPool`、`CombatDataBridge`；`UIStackManager` 与 `SkillSelectPanelController` 在 UI 控制层，经 `SyncSystem` 读写 ECS 而不直接改实体节点；`ConfigBootstrap`、`RunMapGenerator` 在领域服务层；`EcsWorld`、`defines`、对象池与 `Worker` 协议在基础设施层。



图 4-1 系统分层架构

如图 4-1 所示，表现层只负责 Laya 节点与 HUD 显示，不包含技能冷却等业务规则；游戏逻辑层 bulk 更新实体组件，是第 6 章对象池（图 5-2）与 Worker（图 5-4）优化的作用域；领域服务层把配表、跑图生成与战斗入口隔开，便于第 5.4.1 节对照图 5-4 读启动时序。

4.2.1 基于 ECS 的 Gameplay 实现

局内玩法走 ECS。EntityId 是数字；ComponentStore 用 Map 按类型存组件；EcsWorld 按 input/logic/render 分组调 System。Position、Velocity、Attribute、Skill 等只放数据；MovementSystem、BulletSystem 等遍历实体改字段，本身不记状态。

标签与筛选：玩家与怪物共用 Position、Attribute、Skill 等组件，只靠 PlayerTag 或 MonsterTag 区分身份。FilterRegistry 预先注册 Players、Monsters 等筛选名，各 System 按名取实体，不必各自写组件交集查询。扩展新怪物行为时，优先增组件字段或增 System，不维护 MonsterBase 继承树，符合 ECS 中组合优于继承的做法。

战斗数据流：每帧 CombatDataPrepareSystem 先采坐标等快照；实体够多时，经 CombatDataBridge 把快照送 Worker 跑 processCombatFrame；主线程 MonsterChaseSystem 再把追逐、分离结果写回速度；BulletSystem 仍在主线程做碰撞，因为要读 Laya 节点和阵营。具体流程可参考图 4-2。

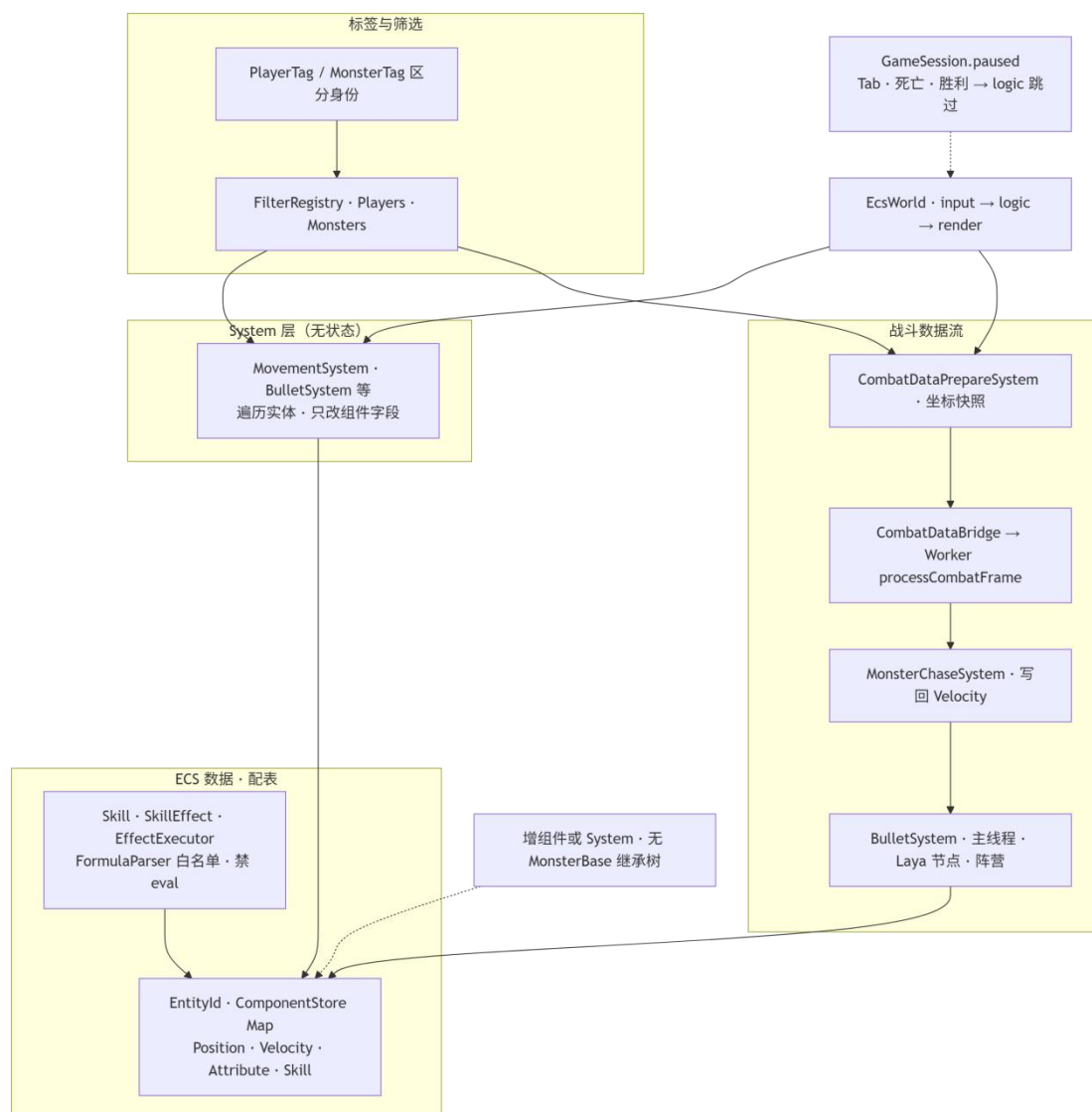


图 4-2 Gameplay 实现图

死亡、打开技能 Tab 或战斗胜利时，将 `GameSession.paused` 置真，逻辑类 `System` 在 `update` 入口直接返回，本帧不再推进战斗。

配表驱动：Skill 一行管冷却，SkillEffect 多行管效果链；EffectExecutor 依次解释 bullet、modifier_*、direct_damage。要给后面的子弹加修饰，修饰行须写在该 bullet 之前，顺序即组合语义。伤害表达式交给 FormulaParser 在白名单内计算，业务代码禁止 eval。

4.2.2 基于 MVC 的 UI 结构实现

元游戏与局内 UI 采用 MVC 变体^[17]：

Model: hp、装配来自 ECS；跑图用 RunMapPanelModel、RunMapState；

View: 预制体、UI2 节点负责布局;

Controller: `UINavigationController` 子类管生命周期和路由，不得直接改战斗实体。

全屏页走 UINavigationController: push 显示、pop 返回，始终只有一个全屏页在前台。

MetaMenuBootstrap 注册开始、选关、跑图路由。

局内 Tab 由 SkillSelectPanelController 监听：打开时 GameSession.paused=true，战斗 System 不 tick，UI 仍能吃点击。拖拽走 SkillDragService，落点用 SkillSlotHitTest；关面板时 SkillLoadoutSyncSystem 一次性写回 Skill 和 PlayerAutoCastSystem。

跑图连线由 RunMapPanelView 画，能不能走由 Model 判，View 不写规则。

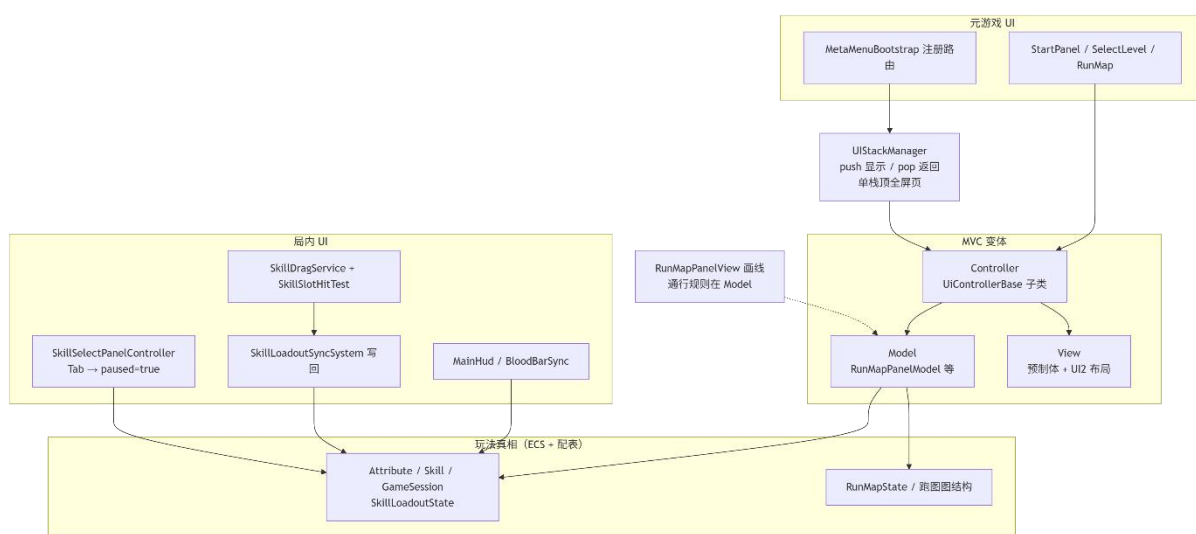


图 4-3 UI 结构图

核心约束：界面不拥有玩法真相——玩法状态以 ECS 与配表为准，UI 仅经 Controller/SyncSystem 读写。

4.2.3 基于 Web Worker 和对象池的优化实现

子弹与怪物分别由 `BulletPool`、`MonsterPool` 管理。池按预制体路径分桶，节点用完回池而不是销毁，从而减少 `instantiate/destroy` 带来的 GC 抖动。`MonsterRecycleSystem` 在怪物与玩家距离超过设定阈值时回收实体，控制场上活跃怪物数量。象池：`BulletPool`、`MonsterPool` 按预制体路径分桶复用 Laya 节点，抑制 `instantiate/destroy` 触发的 GC 尖峰。`MonsterRecycleSystem` 在怪物远离玩家超过阈值后回收实体，维持活跃集合规模。

场上实体数达到 COMBAT WORKER MIN ENTITY COUNT 后, CombatDataPr

epareSystem 把一帧快照交给 CombatDataBridge，再 postMessage 到 Worker。追逐、分离、摆动写在 combatWorkerLogic.ts 里，主线程与 Worker 共用同一份逻辑文件，避免两套算法各写一遍。每帧 CombatDataBridge.prepareFrame 先取 Worker 异步结果；取不到或算失败就走 computeSync，在主线程补算同一帧。子弹碰撞、连锁、分裂仍放在主线程，因为要读 Laya 节点并做当帧命中判定，Worker 里没有这套对象。流程参考图 4-4。

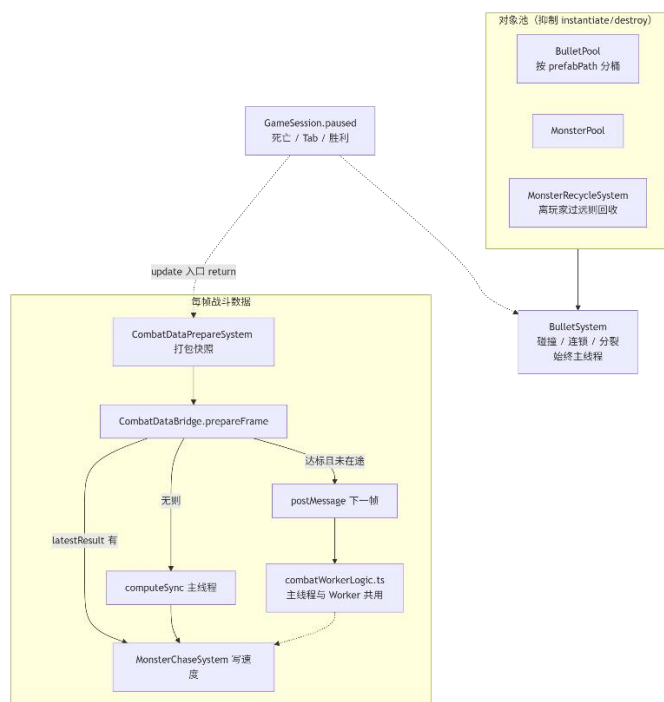


图 4-4 性能优化图

技能面板打开、玩家死亡或胜利时，GameSession.paused 置为真，各战斗 System 在 update 开头直接 return，本帧不再跑战斗逻辑，也不为此多开对象。做法上接近 Cantrell 等把重算与画面更新分开处理的做法^[5]。

4.3 系统交互与时序

从启动到重开一局，主路径上的模块会按固定顺序协作，整体关系如图 4-5 所示。Main.onStart()里先取 Area2D 当 container，再 new InputService、ControlInputAdapter、SimpleEcsDemo，注册 frameLoop 后调 loadConfigAndInitDemo()；该函数首行 await ensureGameConfigLoaded()，对应图中 Main 与 ConfigBootstrap 之间的第一条消息。若 META_MENU_ENABLED 为真，Main 走 startMetaMenu()，container.visible=false 并由 U

IStackManager push 开始界面，对应图中「元菜单开启」分支；否则直接 demo.init()。玩家从跑图或选关确认后，MetaFlowController 触发 onEnterCombat，Main 显示战斗 Area2D 并 init Demo，对应图中「进入战斗」色块。此后每帧 frameLoop 驱动 EcsWorld.update，CombatDataPrepareSystem 在实体够多时经 CombatDataBridge 把快照 postMessage 给 Worker，失败则 computeSync，对应图中「战斗帧循环」里的 alt 分支。按 Tab 装配技能时 SkillSelectPanelController 把 GameSession.paused 置真，对应图中「Tab 技能装配」色块，与第 5.2.2 节图 5-2 展开同一步骤。玩家死亡后 RestartPanel 与 tryRestartFromSession 清场重生，对应图末「死亡与重启」。

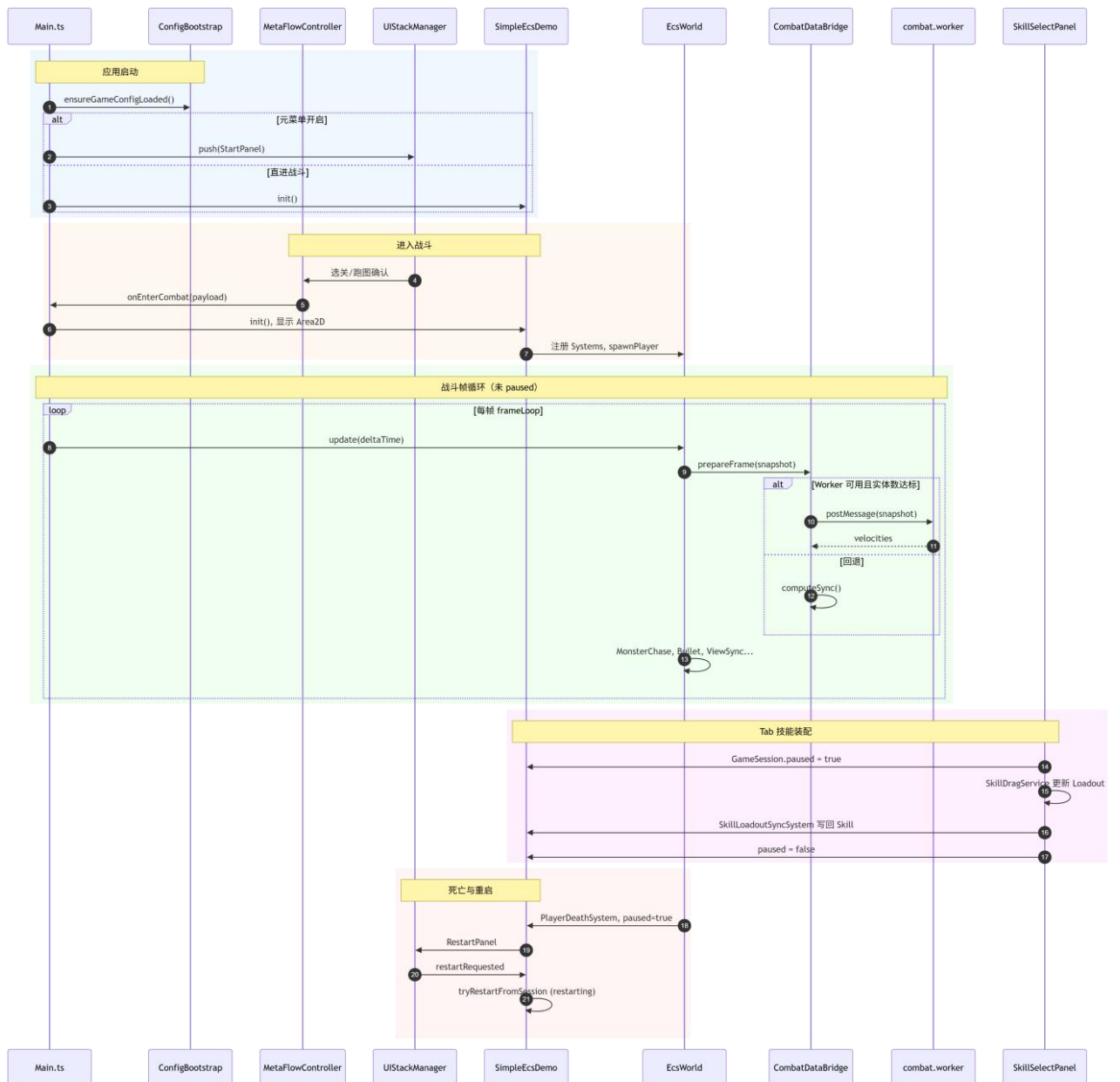


图 4-5 系统主要交互时序

4.4 可靠性设计

配表坏了、Worker 起不来、地图生成失败时，系统应尽量还能玩或至少能调试，而不是白屏退出。做法如下。

(1) 配表加载与就绪。启动时 `ConfigBootstrap.ensureGameConfigLoaded` 按 `configBases()` 列出的路径，对 `tables.registry.json` 及各张表依次用 `fetch` 和 `Laya.loader` 两条路加载。`isGameConfigReady()` 在 `configLoaded` 为真且 `Data.Skill` 已有数据时返回真，作为能否进战斗的总开关。`Main.onStart` 和 `SimpleEcsDemo.spawnPlayer` 在刷玩家前都会等或再查一次该状态。

(2) 运行中缺表。`SimpleEcsDemo` 用 `missingConfigLogged` (Set) 记已报过的键，`logMissingConfigOnce` 对每个缺失键只打一次日志，避免控制台刷屏。查表失败时回退到 `defines` 里的 `DEFAULT_PLAYER_HP`、`DEFAULT_PLAYER_PREFAB`、`DEFAULT_PLAYER_AUTO_SKILL_ID` 等默认值。升级奖池表为空时，代码里给一条内联默认奖励，演示流程仍能往下走。

(3) Worker 主备两条路。`initWorker` 失败、Worker `onerror` 或 `postMessage` 抛错时，把 `workerUsable` 设为 `false`。`prepareFrame` 拿不到异步结果就 `computeSync`，与 Worker 一样调 `processCombatFrame`。`MonsterChaseSystem` 没有 Worker 输出时，在本线程做追逐和分离。`BulletSystem` 遇到连锁弹等 Worker 未实现的类型，整段逻辑仍在主线程跑。实体数低于 `COMBAT_WORKER_MIN_ENTITY_COUNT` 时不往 Worker 送数据，省掉序列化和 `postMessage` 成本。

(4) 跑图保底。`RunMapGenerator.generate` 随机生成最多试 `RUN_MAP_GENERATION_MAX_RETRIES` 次；若 `validateActReachBoss` 仍判 Boss 不可达，就调 `generateFallback`，为三幕各生成一条「休息→战斗→Boss」的线性链，保证 Boss 能走到，局外进度不会卡死。

(5) 重启防重入。`RestartPanelSystem` 将 `session.restartRequested` 置真；`SimpleEcsDemo.tryRestartFromSession` 在 `restarting=true` 期间异步清场与重生，`update` 中避免重复触发，`finally` 复位标志，防止监听器与实体重复注册。

(6) 预制体加载失败。实例化失败时 `createPlaceholderNode()` 生成占位碰撞体，便于在无美术资源环境下继续调试逻辑。

4.5 容错与降级策略

如图 4-6 所示，流程自故障检测起，经分支选择执行对应降级，再回到可运行态，形成闭环。配表加载失败、Worker 不可用、跑图生成失败、预制体缺失等情形各走独立回退支路，避免整局白屏退出。与局内状态相关的路径另作区分：死亡、技能 Tab 装配与战斗胜利等情形共用 `GameSession.paused` 冻结战斗 tick，`RestartPanelSystem` 仅在 `isPlayerDead()` 为真时弹出重启面板，以免装配暂停被误判为失败局。

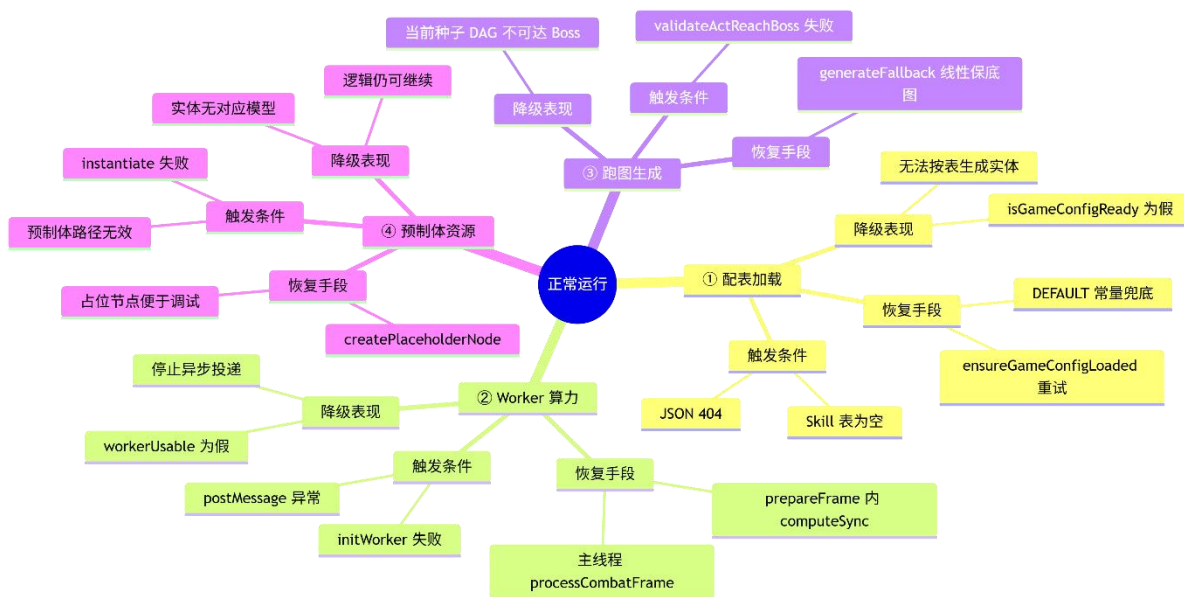


图 4-6 降级策略示意图

4.6 可扩展性设计

本系统扩展新内容时，优先加组件和 `System`，而不是为每种怪或每种技能写子类。做法符合 ECS 里组合优于继承的惯例。

实体侧：新机制先加组件类型（例如以后的 `BuffComponent`、`ShieldComponent`），再写单独 `System` 读写。带 `MonsterTag` 的实体已经进 `FilterRegistry.Monsters`，加组件即可，不用动 `EcsWorld` 内核。无需修改 `EcsWorld` 内核。

行为侧：新逻辑对应新 `System`，或在现有 `System` 里加分支。例如加光环：实现 `AuraSystem`，在 `SimpleEcsDemo` 构造函数里 `addSystem` 即可，`MovementSystem` 可以不动。

技能侧：新 `effect` 在 `EffectExecutor` 里加 `case`；修饰器照旧是「先写上下文、后面的 `bullet` 行再读」。策划在 `SkillEffect` 表里调行顺序就能拼效果。新子弹形状只要在

Bullet 表加一行并指定预制体路径。

内容侧：新节点类型改 RunNodeType 和 RunMapGenerator 的权重表；新全屏页在 UIStackManager 注册路由和 Controller，战斗 ECS 不用改。

若用 Player extends Character extends Actor，每加一种会分裂的精英怪都要子类并重写 update，Laya 节点和逻辑会绑死。现方案里精英怪仍是 MonsterTag，再加 Attribute、MonsterDef.waveWeight 等组件；分裂、自动施法由 MonsterAutoCastSystem 等系统组合出来。这与 Gregory 及 ECS 文献里强调的数据导向、按需增量扩展一致。

4.7 本章小结

本章先写设计原则和分层，再分三块讲具体设计：ECS 玩法、MVC 界面、Worker 与对象池。后面补了端到端时序、可靠性措施、容错表，以及靠组件组合做扩展的做法。以上内容给第 5 章对照源码时当骨架用。

第5章 核心模块详细设计与实现

第5章写各核心模块的实现，对照第4章设计，并引用 src/下源码。SimpleEcsDemo 充当组合根：在这里注册 System、对象池 UI 控制器。

5.1 ECS 架构 Gameplay 实现

局内玩法的调度入口是 EcsWorld，它把 EntityManager、ComponentStore 和 System 注册收在一个门面里，主循环里调用一次 update(deltaTime)就能跑完本帧逻辑。EntityManager 用自增方式发 EntityId，销毁后编号进空闲列表供后续复用。ComponentStore 按组件类型各维护一张 Map，键是 EntityId，System 用 getAllOfType 或 getEntitiesWith 查询时不必扫整张实体表。SimpleEcsDemo 构造函数里会先 this.filters = new FilterRegistry(this.world)，再 registerNamedFilters(this.filters)，并 world.createEntity()后给 session Entity 挂上 GameSession 组件，全局暂停和重启标记都写在这个会话实体上。各 System 在 setupSystems()里 registerSystem，组序和 priority 由 EcsWorld.sortSystems()排好。MonsterChaseSystem、BulletSystem 通过 filters 按 Players、Monsters 等名字取实体，不必在每个 System 里重复写组件交集。销毁怪物时调 world.destroyEntity(id)，内部先 removeAllComponents 再 destroyEntity，与需求章描述一致。

属性、移动和自动施法分给几个 System 各管一块。AttributeSystem 按来源增删 modifier，升级奖励和 Buff 通过它写回 Attribute，结算时可用 getModifierContributions 追到来源。MovementSystem 根据 Velocity 改 Position，ControlSystem 从 ControlInputAdapter 读方向写进 Control 组件，玩家本帧位移先在这里落地。PlayerAutoCastSystem 为三个装备栏各维护冷却和 pendingCasts，施放前检查对应 Skill.cooldownRemain，栏位之间互不抢冷却。MonsterAutoCastSystem 按怪物配表触发敌方技能，和玩家共用 EffectExecutor、BulletSystem 那条链，只是实体带 MonsterTag 而不是 PlayerTag。

玩家侧一次自动施法时，PlayerAutoCastSystem 先从 SkillLoadoutState 读栏位技能 id 并检查 Skill.cooldownRemain，通过后由 buildSkillCastPlan 按配表行顺序遍历 effectIds；modifier 行只累加 pending 变量，bullet 行才把 pending 写入 BulletSpawnSpec 并调用 spawnBulletWithOptions，direct_damage 行把 paramsFormula 交给 FormulaParser 建树求值（细节见图 5-5）。具体分支与先后关系如图 5-1 所示，表中 effect 字段与执行器的对应仍见表 5-1。

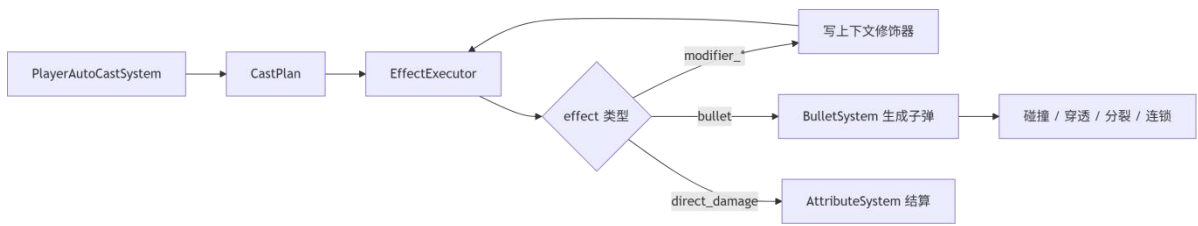


图 5-1 技能效果执行链

如图 5-1 所示，流程从 PlayerAutoCastSystem 检查冷却开始，进入对 effect 行的循环：图中 modifier_split、modifier_chain、modifier_pierce 三个分支只更新 pendingSplit、pendingChain、pendingPierce 并回到循环，不直接生成子弹，这与源码里 continue 不写 plan.bullets 一致；bullet 分支把 pending 写入 splitCount、chainCount、penetration 后调用 BulletSystem.spawnBulletWithOptions，对应 spawn 后主线程碰撞链；direct_damage 分支则走 FormulaParser 扣血。策划若把 modifier 行放在 bullet 行之后，图中 pending 无法被消费，局内就会出现“有修饰无弹幕”的配置错误，因此表内行序与图 5-1 自上而下顺序必须一致。

表 5-1 效果类型与执行器映射

effect 字段	行为
bullet	BulletSystem.spawnBulletWithOptions
modifier_split	设置分裂数量
modifier_chain	设置连锁与半径
modifier_pierce	增加穿透
direct_damage	公式直伤

子弹和怪物 AI 是主线程里最吃 CPU 的两块。BulletSystem 生成子弹时先调 BulletPool.get(prefabPath)，从 Map 里对应数组 pop 节点，数组空则返回 null 再由 BulletSystem instantiate；回收时 BulletPool.put 会在 node.parent 存在时 removeChild，再把节点 push 回桶。飞行中用 BulletHitTest 的 hitRadiusSq 做圆碰撞，命中后扣血、递减穿透，并按配表做连锁或分裂。MonsterWaveSpawnSystem 在环带刷怪，MonsterChaseSystem 每帧算追逐并叠分离力和摆动；若 CombatDataBridge 本帧有 latestResult，就 applyWorker Velocities 把 vx、vy 写回 Velocity，否则在主线程用 processCombatFrame 同一套逻辑。离屏或过远的怪由 MonsterRecycleSystem 收回 MonsterPool，MonsterContactDamageSystem 处理贴脸伤害。

经验和元游戏数据把局内成长和局外路线接在一起，ExperienceSystem 在击杀时加经验并判断升级，UpgradeRewardSystem 拉起奖励面板（界面见图 7-5），RewardPoolService 按权重抽条目，RewardApplyService 写回 Attribute 或技能相关组件。RunMapGenerator 生成三幕 DAG，随机生成与保底回退如图 5-3 所示，validateActReachBoss 过不了就走 generateFallback。MetaFlowController 在用户确认节点后触发 onEnterCombat (payload)，Main 显示战斗容器并 await SimpleEcsDemo.init()，payload 里可带 Boss 胜利时长等参数，对应 5.5.1 场景从菜单进战斗的路径。

5.2 MVC 架构 UI 实现

MetaMenuBootstrap 向 UIStackManager 注册 StartPanel、SelectLevelPanel、RunMapPanel 等路由。用户确认节点后触发 onEnterCombat，Main 隐藏菜单容器、显示战斗 Area2D 并 demo.init()。RunMapPanelView 使用 MapNodeIconUtil、LineLayoutUtil 渲染 DAG；仅允许选择已解锁邻接边。

局内 HUD 和技能装配是玩家最常接触的 UI，MainHudSystem、BloodBarSyncSystem 把 hp、maxHp 同步到 ProgressBar。Tab 打开装配面板时的消息顺序如图 5-2 所示，运行界面见图 7-4。SkillSelectPanelController 在 registerTabKey()里监听 TAB_KEY_CODE，keydown 时调 togglePanel()；setPanelVisible(true)会把 session.paused 设为 true 并回调 onTabPauseChange，战斗侧 isPaused()使各 System 在 update 开头 return。长按超过 LONG_PRESS_MS 后 SkillDragService 顶层代理拖拽，SkillSlotHitTest 区分装备栏与效果槽；关面板时 SkillLoadoutSyncSystem 写回 Skill 与 PlayerAutoCastSystem，getCombatEffectIds 刷新后新 effect 链在下一冷却周期生效。

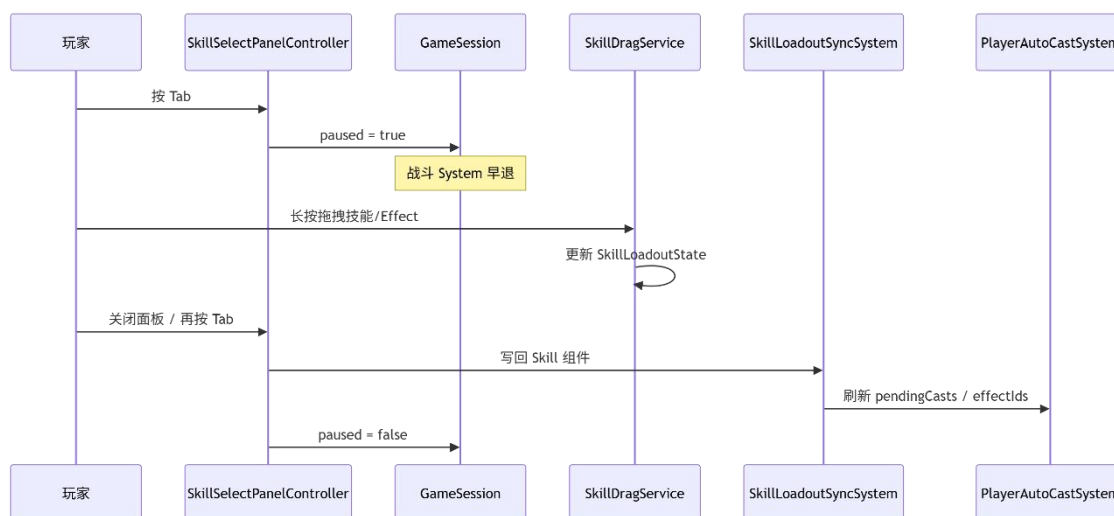


图 5-2 Tab 技能装配交互时序

如图 5-2 所示，玩家按 Tab 后 UI 先把 `GameSession.paused` 置真，图中「战斗 System 早退」对应第 5.4.3 节帧循环描述；拖拽阶段只改 `SkillLoadoutState`，图中 Drag 自环表示 `SkillDragService` 在面板打开期间反复刷新栏位；关闭面板时 Sync 写回 Skill 组件并刷新 `PlayerAutoCastSystem`，图中 Cast 参与下一周期施放，与图 5-1 顶端 `cooldown` 检查相连。图 4-2 中「Tab 技能装配」色块是本图的缩略，两图可并置说明 UI 与战斗 tick 的切断关系。

输入和战败重启在工程里分成三条线，和图 4-2 末尾「死亡与重启」色块、图 5-2 里「`paused` 但 UI 仍响应」的设定是对得上的。

移动输入这条线，`Main.onStart` 会 new `InputService` 并包一层 `ControlInputAdapter` 传给 `SimpleEcsDemo`，`ControlInputAdapter.getMoveAxis()` 读 WASD 是否按下，算出归一化前的轴向量，`ControlSystem` 在 input 组里把轴乘 `PLAYER_MOVE_SPEED` 写进玩家 `Velocity`；`ControlSystem` 构造时注入了 `()=>this.isPaused()`，一旦 `GameSession.paused` 为真就直接 return，所以 Tab 打开技能面板时玩家不会继续位移，但 `SkillSelectPanelController` 仍挂在 `document` 或 `Laya.stage` 上收 Tab 和拖拽，不会和 `ControlSystem` 抢同一条输入通道。

战败这条线由 `PlayerDeathSystem` 和 `RestartPanelSystem` 配合完成。`PlayerDeathSystem` 在 logic 组 priority 为 -10，每帧看 `Players` 筛选器里玩家 `Attribute.base.hp`，若 `hp ≤ 0` 就把 `hp` 钳到 0，并把 `session.paused` 与 `session.combatFailed` 置真，对应图 4-2 里战斗 tick 停住、`Main.handleCombatFailed` 停生存计时。`RestartPanelSystem` 跑在 render 组，它用 `isPlayerDead()` 区分「Tab 装配暂停」和「真死亡」：只有 `paused` 且 `hp ≤ 0` 才 `uiStack.push(RESTART_PANEL_ROUTE_ID)`，避免装配面板误弹重启，界面见图 7-6；玩家点重启后回调里写 `session.restartRequested=true`，`SimpleEcsDemo.update` 里 `tryDefeatExitFromSession` 在 `restarting` 为 true 时调用 `Main.onDefeatExitRequested`，finally 一定把 `restarting` 复位，避免重复清场或监听器叠两层。

依赖关系仍保持单向：UI 经 `Controller`、`SyncSystem` 读写 ECS，`System` 只读配表和组件，不在 `System` 里直接操作 UI 节点，和第 4 章 MVC 变体一致^[17]。

5.3 Web Worker 和对象池的优化实现

第五章把第 4.2.3 节的池化和 Worker 设计落到具体类上，建议读者把本节与图 5-3、

图 5-4 对照看（两图读图段见本节末），再回头看 SimpleEcsDemo 构造函数里 bulletPool、combatBridge 的注入关系。

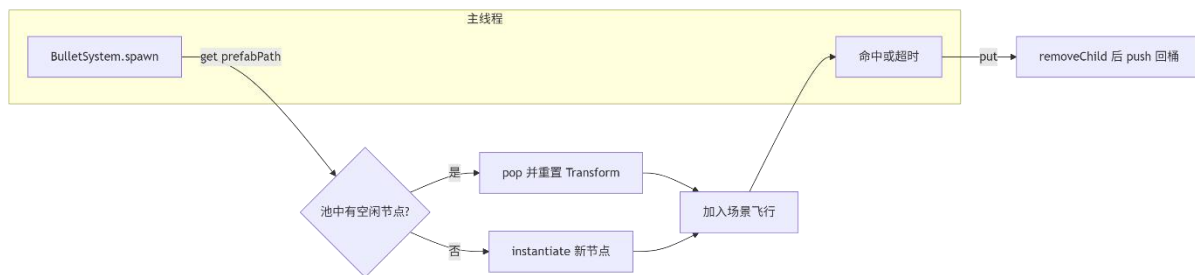


图 5-3 对象池示意图

对象池侧，SimpleEcsDemo 在构造时 new BulletPool()和 MonsterPool()，new BulletSystem 时传入() $\Rightarrow$ isObjectPoolEnabled()。BulletSystem 要生成子弹时先 bulletPool.get(prefabPath)，get 内部对 Map 里对应数组做 pop，拿不到节点才走 Laya 预制体 instantiate；子弹寿命结束或穿透耗尽后 put 回池，put 会先把 node 从 parent removeChild 再 push 进桶，和图 5-3 里「spawn $\rightarrow$ 飞行 $\rightarrow$ 回收」闭环一致。MonsterRecycleSystem 判定怪物离玩家过远后同样 put 进 MonsterPool，而不是 destroy，这样千怪波高峰过后数组里会攒一批可复用节点，对应第 6 章表 7-6 里开池后 P95 下降、均值 FPS 几乎不变的现象。

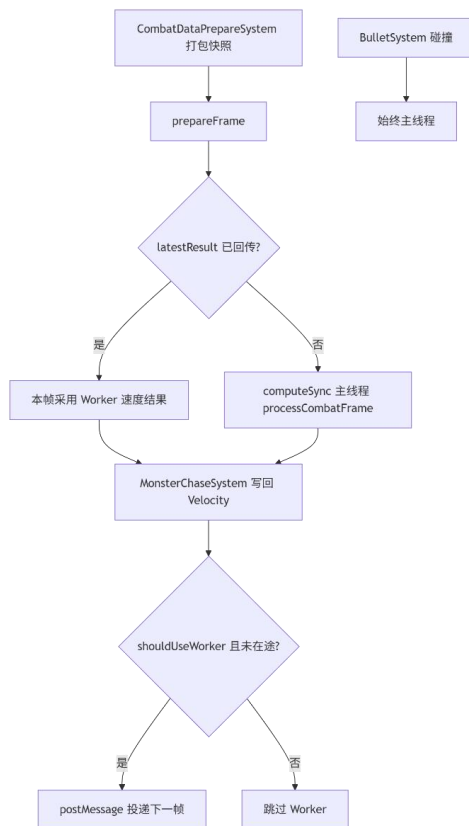


图 5-4 WebWorker 流程图

Worker 侧，CombatDataPrepareSystem 在 isPaused()为假时从 Filters 取怪物坐标，拼成 CombatEntitySnapshot，再调 combatBridge.prepareFrame。prepareFrame 的逻辑在图 5-4 里画得很清楚：本帧若已有 latestResult 就消费 Worker 上一帧算好的速度，否则在主线程 computeSync 调 processCombatFrame；帧末 dispatchCompute 在 shouldUseWorker 为真且没有 pending 请求时 postMessage 给 Worker。shouldUseWorker 会看 entityCount+bulletCount 是否达到 COMBAT_WORKER_MIN_ENTITY_COUNT，消融实验里 MetaRunSession.testUseCombatWorker 为 false 时整条 Worker 支路被关掉，方便和第 4、5 波「无 Worker」同窗对比。MonsterChaseSystem 只负责把算出的 vx、vy 写回 Velocity；BulletSystem 的 hitRadiusSq 碰撞、连锁、分裂仍留在主线程，因为必须读 Laya 节点和阵营，图 5-3 最下方「始终主线程」支路与 5.1.4 节文字一致。主线程与 Worker 共用 combatWorkerLogic.ts，避免两套追逐公式。

5.4 游戏主循环设计

Main.onStart 创建 InputService、ControlInputAdapter、SimpleEcsDemo，注册 frameLoop，并 await ensureGameConfigLoaded()。配表优先于实体生成，避免 Data.Character.GetByID 为空。若 META_MENU_ENABLED 为真则 startMetaMenu，否则直接 demo.init()。

SimpleEcsDemo 在工程里当组合根，构造函数里 this.bulletPool=new BulletPool()，this.combatBridge=new CombatDataBridge()，new BulletSystem 时把 bulletPool 和()=>isPaused()、()=>isObjectPoolEnabled()传进去，new CombatDataPrepareSystem 并把 combatPrepare 设回 BulletSystem，再注册 ExperienceSystem、SkillSystem、MonsterWaveSpawnSystem、UpgradeRewardSystem、MainHudSystem 等，skillSelectController.setOnTabPauseChange 把 Tab 暂停通知 Main。init()在配表就绪后按 Character 玩家行加载预制体，给 playerEntity 挂 PlayerTag、Position、Attribute、Skill、SkillLoadoutState、ViewComponent 等，怪物交给 MonsterWaveSpawnSystem 按波次在环带生成，不在 init 里手写每只怪。

帧循环在 Main 与 SimpleEcsDemo 之间拆成两层，和图 4-2「战斗帧循环」色块、图 5-2 里 paused 切断战斗 tick 的说法一致。

Main 侧，Laya.timer.frameLoop(1, this, onFrameLoop)每帧先 input.update()刷新按

键状态，再 `demo.update(deltaTimeSeconds)`，最后 `updateCameraFollow` 按玩家 `Position` 平滑移动 `Area2D` 根节点，相机跟随不写在 ECS 里，避免 `System` 去碰 `Laya` 容器坐标。`Main` 还通过 `setOnTabPauseChange` 接 `Demo` 回调：`Tab` 打开时 `combatSurvivalTimer.pause()`，关闭时 `reset`，这样装配技能时生存倒计时也会停，但 ECS 里仍只靠 `GameSession.paused` 冻战斗逻辑。

`Demo` 侧，`SimpleEcsDemo.update` 在测试模式下会先 `testLevelFpsTracker.tick`，然后无条件 `world.update(deltaTime)`——注意 `paused` 并不是在 `world.update` 外包一层 `if`，而是各个战斗 `System` 构造时注入 `isPaused`，在各自 `update` 开头 `return`，所以图 5-2 写「战斗 `System` 早退」时，`frameLoop` 其实仍在跑，只是 `logic` 组里追逐、刷怪、子弹等步骤空转。`update` 末尾若 `restarting` 为 `false` 会 `tryDefeatExitFromSession` 处理 `restartRequested`；`restarting` 为 `true` 时跳过，防止战败退出重入。`PlayerDeathSystem`、`SkillSelectPanelController`、胜利奖励等都会写 `session.paused`，ECS 层只认这一枚冻结位，避免 UI 和玩法各维护一套 `pause` 标志。

5.5 典型场景实现剖析

场景一：玩家第一次进入流程

玩家第一次进战斗时，流程如图 4-2 中「应用启动→进入战斗」两段所示：`Main.onStart` 触发 `loadConfigAndInitDemo` 并 `await ensureGameConfigLoaded()`，元菜单模式下 `UIStackManager` `push StartPanel` 且战斗 `Area2D` 不可见，界面见图 7-1。玩家确认跑图或选关节点后 `MetaFlowController` 调 `onEnterCombat`，`Main` 显示战斗容器并 `await demo.init()`，`CombatSurvivalTimer` 开始计时，对应图 4-2 中 `Demo` 注册 `Systems`、`spawnPlayer`。首帧 `world.update` 时 `ControlSystem`、`PlayerAutoCastSystem`、`MonsterWaveSpawnSystem` 并行，即图 4-2「战斗帧循环」内第一次 `tick`。功能验证对应 T-F-01、T-F-09；局内战斗画面见图 7-3。

场景二：`Tab` 打开技能面板并更换装备

玩家进局后按 `Tab`，`SkillSelectPanelController.registerTabKey` 在 `keydown` 里发现 `keyCode` 等于 `TAB_KEY_CODE` 就 `togglePanel`，`setPanelVisible(true)` 会把 `SkillLoadoutState.panelOpen` 设为 `true`，并从 `sessionEntity` 取 `GameSession` 把 `paused` 写成 `true`，`onTabPauseChange(true)` 传到 `Main` 去 `pause` 生存计时；各战斗 `System` 因 `isPaused()` 在 `update` 入口 `return`，怪物不再追、子弹不再出新，但 UI 仍能收到指针事件。玩家长按超过 `LON`

G_PRESS_MS, SkillDragService 在顶层挂代理图标, SkillSlotHitTest 在 panelRoot 坐标系里判断落点是装备栏还是 effect 槽, SkillLoadoutState 里栏位数据被改掉但尚未写回 Skill 组件。玩家再按 Tab 或关面板, setPanelVisible(false)触发 SkillLoadoutSyncSystem, 把 loadout 一次性同步到 Skill 和 PlayerAutoCastSystem, getCombatEffectIds 刷新后 paused 恢复 false; 下一周期 Cast 参与施放, 实际伤害要等 cooldownRemain 走完才走图 5-1 的 buildSkillCastPlan 路径。面板实拍见图 7-4。

场景三：怪物密集时的 Worker 卸载

实场景三用 Level3 千怪波说明 Worker 支路何时生效、何时必须回退主线程, 单帧分工如图 5-3 所示。

同屏怪物数和子弹数加起来达到 COMBAT_WORKER_MIN_ENTITY_COUNT, 且 GameSession.paused 为假时, CombatDataPrepareSystem 会把玩家与怪物坐标打进 CombatEntitySnapshot, 调用 CombatDataBridge.prepareFrame。若本帧 latestResult 已从 Worker 回传, MonsterChaseSystem 走 applyWorkerVelocities 写速度, 否则 prepareFrame 内部 computeSync 在主线程跑 processCombatFrame, 和 Worker 共用 combatWorkerLogic.ts。帧末 dispatchCompute 再 postMessage 投递下一帧。与此同时 BulletSystem 仍在主线程做 hitRadiusSq 碰撞和连锁。若在 MetaRunSession 里关 testUseCombatWorker, 表 7-5、7-6 第 4、5 波数据会体现这条边界。

如图 5-3 所示, 左侧「打包快照」对应 CombatDataPrepareSystem, 菱形「latestResult 已回传」决定本帧消费 Worker 还是 computeSync, 下方「始终主线程」支路说明子弹碰撞不进 Worker;

5.6 关键代码文件索引

表 5-2 核心源码文件索引

模块	路径
入口	src/Main.ts
ECS 世界	src/ecs/core/World.ts
组件存储	src/ecs/core/ComponentStore.ts
战斗 Demo	src/game/demo/SimpleEcsDemo.ts
子弹	src/game/bullet/BulletSystem.ts
子弹池	src/game/bullet/BulletPool.ts
效果执行	src/game/skill/EffectExecutor.ts

续表 7-5

模块	路径
跑图生成	src/game/run/RunMapGenerator.ts
元流程	src/game/meta/MetaFlowController.ts
UI 栈	src/game/ui/mvc/UIStackManager.ts
技能面板	src/game/ui/skillselect/SkillSelectPanelController.ts
配表引导	src/config/ConfigBootstrap.ts
静态常量	src/defines/*.ts

5.7 功能模块与论文章节对照

表 5-3 功能模块与论文章节对照

功能模块	论文章节
ECS Gameplay（实体、技能、子弹、怪物）	第 5 章第 5.1 节；第 4 章第 4.2.1 节
MVC UI（栈、跑图、技能装配）	第 5 章第 5.2 节；第 4 章第 4.2.2 节
Web Worker 与对象池	第 5 章第 5.3 节；第 6 章
主循环与入口	第 5 章第 5.4 节
可靠性、容错与可扩展性	第 4 章第 4.4 - 4.6 节
测试与性能实验	第 7 章

5.8 本章小结

本章按四条线写实现：ECS 玩法、MVC UI、Worker/对象池、主循环。另用三个场景把配置加载、流程切换和战斗跑通。代码里用到的结构包括：SimpleEcsDemo 作组合根，对象池，CombatDataBridge 作桥，Targeting 作策略，GameSession.paused 作状态开关；没有再加一层通用框架。

实现里遇到过四类问题。（1）配表没拷到 bin/config 会 404：用 sync-config-to-bin.ps1 同步，并用 isGameConfigReady 挡在未就绪前进战斗。（2）UI2 拖技能槽点不中：顶层代理加 SkillSlotHitTest。（3）Effect 行顺序错会算错：靠文档和样例表约定顺序。（4）Worker 与主线程结果要对齐：共用 combatWorkerLogic.ts，异步失败时 computeSync。表 5-2、表 5-3 把模块和文件对应起来。性能测量放在下一章。

第 6 章 性能优化与工程实践

本章对照第 4.2.3 节设计，写清已落地的优化、实验步骤和工程习惯，并与第 7 章数据对齐。原则是先量后开：池和 Worker 都能关，方便对照和排错。

6.1 性能目标与瓶颈

性能目标定在 1920×1080、IDE 内置 Chromium 预览下，Level3 第 1、2 波怪物较少时 TestLevelFpsTracker 统计的平均 FPS 尽量贴近 60，第 3 波及千怪消融段则重点看 P95 帧时间（表 7-5、表 7-6），因为均值容易被低负载波次拉高，掩盖偶发尖峰卡顿。

瓶颈在工程里大致能对应到三类开销：一是 MonsterChaseSystem 与 BulletSystem 的 CPU，同屏实体上千时追逐、圆碰撞、连锁搜索都会上去；二是 BulletPool、MonsterPool 若关掉，spawn 路径会频繁 instantiate 和 destroy，浏览器 GC 在 Performance 面板里表现为 Scripting/GC 尖刺，表 7-6 第 4 波无池时 P95 到 76.50 ms 就是这类问题的体现；三是主线程还要跑 Laya 绘制和 UI，千怪千弹时 DrawCall 和贴图采样也会吃时间。

对策与后文图示一一挂钩：图 5-3 说明 get/put 如何把建删变成复用；图 5-3 说明 prepareFrame 如何把追逐从主线程挪到 Worker 又保留 computeSync 回退；图 5-3 与图 4-2 的 paused 段说明 Tab 装配、死亡时各 System 早退，避免无效 tick 白跑；图 6-3 与表 7-4 记录 TextureAtlasService 接入后千怪波 FPS 从约 13.32 提到约 15.10 的变化。

6.2 对象池

对象池实现如图 5-3 所示。BulletSystem.spawn 时先经图中「get prefabPath」判断桶内是否有节点：BulletPool.get 对 arr.pop()，为空则走 instantiate 新节点；子弹命中或超时后 removeChild 再 put 回桶。SimpleEcsDemo 注入的 isObjectPoolEnabled()为 false 时，图 5-3 左侧 get 被跳过，直连 instantiate，即表 7-6 第 4 波「无池」配置。MonsterPool 与 MonsterRecycleSystem 同理。第 4、5 波千怪同窗中开池后 P95 由 76.50 ms 降至 48.60 ms（约 36.5%），平均 FPS 几乎不变，说明池主要削尖峰。

6.3 Web Worker 卸载

追 Worker 卸载只覆盖追逐、分离、摆动等纯数值环节，不涉及 Laya 节点。CombatDataPrepareSystem 在 isPaused()为假且场上实体数达到 COMBAT_WORKER_MIN_E

NTITY_COUNT 时，从 FilterRegistry 取出怪物坐标等字段，拼成 CombatEntitySnapshot 后调用 CombatDataBridge.prepareFrame。

单帧内 prepareFrame 分两步：先消费、再投递。若本帧已有 Worker 回传的 latestResult，MonsterChaseSystem 直接把其中的速度写回各怪物 Velocity；若没有，则在主线程走 computeSync 调用 processCombatFrame，保证不阻塞可玩性。帧末在 shouldUseWorker 为真且无在途请求时，由 dispatchCompute 向 Worker postMessage 投递下一帧计算。initWorker 失败或 postMessage 异常时 workerUsable 置为 false，后续全程退化为 computeSync 单路径。主线程与 Worker 共用 combatWorkerLogic.ts，避免两套追逐公式分叉。

子弹碰撞、连锁、分裂仍由 BulletSystem 在主线程完成，须读取节点位置与阵营，Worker 侧不承载这条路径。实验上，第 1 - 3 波在实体达标时启用 Worker 参与追逐；第 4 - 5 波为对象池同窗对比，第 5 波通过 MetaRunSession.testUseCombatWorker 关闭 Worker 以隔离池化影响。解读表 7-5、表 7-6 时勿与第 3 波冷启动工况混比。

6.4 渲染与逻辑侧其它优化

除对象池与 Worker 外，本工程在碰撞判定、逻辑冻结、绘制合批与配置访问上还有若干针对单帧耗时的细化，与第 5 章实现及第 7 章实验数据相互印证。

碰撞判定上，BulletHitTest 以距离平方比较子弹与怪物中心距，命中分支不做 Math.sqrt，千弹同屏时可减少一批浮点开方，与 BulletSystem 中穿透递减、连锁搜索同属主线程热点，但不进入 Worker。

逻辑冻结上，GameSession.paused 为真时，注入 isPaused 的 System（含 ControlSystem、MonsterWaveSpawnSystem、BulletSystem 等）在 update 入口直接 return。技能 Tab 装配、PlayerDeathSystem 判定阵亡、生存计时胜利弹奖励等情形共用同一枚 paused，避免界面已停而刷怪、碰撞仍在推进的双开关问题；RestartPanelSystem 仅以 isPlayerDead()区分真死亡与装配暂停。

渲染合批上，Main.onStart 通过 installTextureAtlasLifecycle 挂载生命周期，战斗进出时 onCombatEnter/onCombatLeave 配合 TextureAtlasService 对战斗图标做动态合批。表 7-4 显示，第 3 波千怪千弹场景接入动态图集后平均 FPS 由约 13.32 升至约 15.10（约+13.4%），说明该路径主要改善绘制侧开销，与 6.2、6.3 节针对逻辑线程的优化形成互补。对象池消融方面，表 7-6 第 4、5 波同窗对比显示开池后 P95 由 76.50 ms

降至 48.60 ms，平均 FPS 几乎不变，与 6.2 节「削尖峰、稳均值」的结论一致。

配置访问上，`ensureGameConfigLoaded` 仅在启动链读表一次；`MONSTER_COUNT`、`COMBAT_WORKER_MIN_ENTITY_COUNT` 等热路径常量收在 `src/defines`，战斗 tick 内不再反复访问磁盘，减少与玩法无关的 I/O 干扰。

6.5 帧预算与内存

60 FPS 时一帧大约 16.67 ms。粗分：输入和 UI 1–2 ms，ECS（含子弹）6–8 ms，Worker 通信 1 ms 以内，其余给渲染^[7]。超了先开池或 Worker，再降同屏上限，别在业务里硬砍分辨率。

短生命周期对象多会诱发 GC。池化把分配摊平，Performance 面板里 Scripting/GC 段往往跟着好看些。观测靠 `TestLevelFpsTracker`；`missingConfigLogged` 对缺表键只打一次日志。

6.6 性能实验设计

实验要求

看池、图集、Worker 对 FPS 和 P95 的影响。

自变量：池开/关；Worker 开/关（怪够多时）；同屏怪数。

因变量：平均 FPS、P95（消融段）。

控制：同一浏览器、1920×1080、Level3 五波脚本、关后台大程序。

实验步骤：

1. IDE 预览或发布，可选开 Performance；
2. Level3：5 波各 10 s，怪数 10/100/1000，第 4–5 波千怪消融；
3. `TestLevelFpsTracker` 记均值，第 4–5 波记 P95；
4. 对照表 7-5、表 7-6 并写几句解读。

优化自检清单：

合代码前自查：热路径有没有狂 `new`；销毁前有没有 `pool.put`；碰撞是不是平方距离；`paused` 能不能挡住无关 System；Worker 载荷是不是纯数；配表是不是只加载一次。

6.8 本章小结

本章写了池、Worker、碰撞/暂停/图集和帧预算，并给出实验步骤。数据上：池主要救 P95；图集抬千怪波平均 FPS；Worker 在第 1-3 波参与追逐重算。下一章列用例、环境和读表方法。

第 7 章 系统测试与验证

本章用单元脚本、功能回归、性能实验三类手段检查第 3 章需求和第 4 章设计是否落到实处，策略参考 Petty 等对仿真与交互系统 V&V 的做法[16]。单元层验证不启动完整 Laya 场景，适合在改 EcsWorld、FormulaParser 之后快速回归；功能层用 T-F 系列步骤对照图 4-2、图 5-2 的主路径；性能层用 Level3 五波和 TestLevelFpsTracker 生成表 7-5、7-6，与第 6 章图 6-1、图 6-2 的优化叙述交叉验证。

7.1 测试策略

单元层覆盖 `ecsCorePhase1.verify.ts`、`formulaParser.verify.ts`、`attributeModifier.verify.ts`，谁改动就 `rerun` 谁。集成层至少走通三条：`RunMapGenerator` 多随机种子下 `validate ActReachBoss`，失败须 `generateFallback`；Tab 场景对照图 5-2 做 T-F-05/06，看 `paused` 与装配写回；`Worker` 开关前后怪物轨迹是否仍来自 `combatWorkerLogic.ts`。性能层固定 Level3 五波脚本，第 4、5 波同窗对比池化，结合图 6-1、表 7-6 解读 P95。若动过 `FormulaParser`（公式树）、`ComponentStore`、`AttributeSystem`、`ConfigBootstrap` 或 `CombatDataBridge`。

7.2 单元验证

ECS 单元验证：

ECS 单元验证在 `ecsCorePhase1.verify.ts`，脚本搭不依赖 Laya 的最小 `EcsWorld`，检查 `EntityId` 不重复、`destroyEntity` 后 `getComponent` 读不到旧数据、组件增删与 `getAllOf Type` 一致、多 `System` 按分组和 `priority` 的 `update` 顺序与注册表一致。验收项与 `world.destroyEntity` 先 `removeAllComponents` 再 `destroyEntity` 的实现及 `registerSystem` 组序对齐，动过 `World.ts` 或 `ComponentStore.ts` 后应 `rerun`[18]。

公式解析器：

`formulaParser.verify.ts` 覆盖：四则运算、括号、属性别名；非法字符与边界公式（以实现为准的安全返回/抛错）。配表伤害依赖解析器，任何变更须 `rerun`。

属性修饰器：

属性修饰器是连接升级奖励、Buff 与 `FormulaParser` 上下文的关键一环，验证脚本在 `attributeModifier.verify.ts`。

局内 Attribute 组件的 base 里存 hp、atk 等基础值，AttributeSystem 维护 modifier 列表，每条 modifier 带 sourceId 和数值，getFinalValue 在结算时把 base 与叠加结果合成最终属性。UpgradeRewardSystem 发奖励或后续 Buff 写 modifier 时，都经 AttributeSystem 增删；EffectExecutor 构造 direct_damage 的 context 时，正是遍历 casterAttr.base 的键去取 getFinalValue，所以公式树里的 ident 名字必须和 base 字段对得上，否则 evaluateFormula 会抛 unknown identifier。

attributeModifier.verify.ts 会测多个 modifier 叠加后的 final 值、按 sourceId 批量移除是否干净、getModifierContributions 能否列出每个来源的贡献。脚本若失败，局内伤害数字和 BloodBarSyncSystem 刷新的 hp 往往会一起错，和表 3-5 里属性修饰器验收项对应。如图 5-1 的 direct_damage 分支进入 EffectExecutor，context 经 AttributeSystem 展开后由图 5-5 的 evaluateFormula 结算。

7.3 功能测试

表 7-1 列出主要手工用例，走通战斗、元游戏、跑图、装配和重启。

表 7-1 功能测试用例摘要

编号	步骤	预期结果
T-F-01	启动并等待加载	isGameConfigReady，技能表非空
T-F-02	进入战斗移动	角色与相机正常
T-F-03	怪物接近	追逐、扣血、血条更新
T-F-04	观察自动攻击	子弹命中；穿透/分裂/连锁
T-F-05	Tab 打开技能面板	paused=true，可拖拽装配
T-F-06	关闭面板	战斗恢复，下一冷却周期新技能生效
T-F-07	升级	奖励面板与应用
T-F-08	玩家死亡	重启面板，restarting 无重入
T-F-09	元游戏进战斗	生存计时启动
T-F-10	Boss 节点	Boss 胜利时长参数生效

跑图可达性：对 RunMapGenerator.generate 多次随机种子断言 validateActReachBoss；失败应走 generateFallback。

装配同步：互换装备栏技能后，下一冷却周期伤害/图标与装配一致，否则 SkillLoadoutSyncSystem 存在缺陷。

Worker 一致性：Worker 开、关各跑一段，怪物轨迹可差一帧，但算法不能两套^[1]。

7.4 性能测试

7.4.1 测试环境

表 7-2 为五波实验与功能抽测的统一环境（2026-05-18，单次运行）。

选关 Level3 共 5 波：第 1–3 波怪物 10/100/1000，装备三把测试法杖；第 4–5 波均为 1000 怪，依次为「无池无 Worker / 仅池（关 Worker）」，用于同窗消融。第 3 波含首次千怪冷启动，不纳入与第 4–5 波的公平对比。

表 7-2 测试环境配置

项目	配置
操作系统	Windows 10/11（64 位）
运行方式	LayaAir IDE 内置浏览器预览（Chromium 内核）
视口 / 设计分辨率	1920×1080 预览；设计稿 1334×750，showall 缩放
引擎与语言	LayaAir 3.x，TypeScript
测试关卡	选关 Level3：5 波 × 10 s，双方无敌
统计工具	TestLevelFpsTracker（控制台日志）
工程版本	Git d0f08f5（2026-05-21 20:24:57 +0800，build.ps1 生成）

7.4.2 测试结果

表 7-3、7-4 记录接图集前后两轮采样，只看前三波。第 3 波 1000 怪：FPS 从 13.32 到 15.10（约 +13.4%），弹幕多时合批有用。

表 7-3 测试关卡分波 FPS（接入动态图集，前三波）

波次	同屏怪物	单发子弹	平均 FPS	采样帧数
1/3	10	10	56.74	569
2/3	100	100	40.72	408
3/3	1000	1000	15.10	152
前三波汇总	—		37.48	1129

表 7-4 动态图集接入前后 FPS 对比（前三波）

波次（怪物数）	对照 FPS	动态图集 FPS	变化
1（10）	58.68	56.74	-1.94
2（100）	46.30	40.72	-5.58
3（1000）	13.32	15.10	+1.78
前三波汇总	39.46	37.48	-1.98

表 7-5 是同一次跑完的五波，已开图集和 DrawCall 合批。共 1733 帧、约 50.3 s，整场平均 FPS 约 34.48。

结果解读

1.规模曲线（第 1-3 波）。怪物 10→ 100→ 1000 时 FPS 由 58.62 降至 15.14，符合实体与弹幕压力上升规律。

表 7-5 Level3 五波测试帧率（单次运行）

波次	怪物数	池	Worker	平均 FPS	P95 (ms)
1	10	开	开	58.62	—
2	100	开	开	46.84	—
3	1000	开	开	15.14	—
4	1000	关	关	26.16	76.50
5	1000	开	关	25.98	48.60
前五波汇总	—	—	—	34.48	—

表 7-6 性能对比实验（对象池消融，1000 怪/10 s）

配置	平均 FPS	P95 (ms)	对应波次
基线（无池、无 Worker）	26.16	76.50	第 4 波
仅对象池（关 Worker）	25.98	48.60	第 5 波
参考：第 3 波（池 +Worker 全开，首次千怪）平均 FPS 15.14，不宜与上表直接对比。			

2.平均 FPS：池化对均值影响有限。第 4、5 波均值接近（26.16 对 25.98），预热后瓶颈主要在子弹碰撞与绘制。

3.P95。开池后 P95 从 76.50 ms 到 48.60 ms（约降 36.5%），和第 6 章对 GC /实例化的判断一致。

4.第 3 波低于第 4 波基线。第 3 波含资源预热与首次千怪开销；对象池消融结论以第 4-5 波为准。

7.5 系统运行截图

图 7-1-7-6 覆盖元游戏与局内闭环。

7.6 测试结论

功能：Must 用 T-F-01-10 和 verify.ts 覆盖；Should 的跑图、装配、升级、池/Worker 也测过；Could 的法杖三槽 UI 和部分 Effect 特效没做完，和第 8 章一致。

性能：图集让千怪波 FPS 高约 13.4%；五波整场均值约 34.48；同窗消融里池把 P95 拉低约 36.5%。



图 7-1 游戏开始界面（StartPanel）

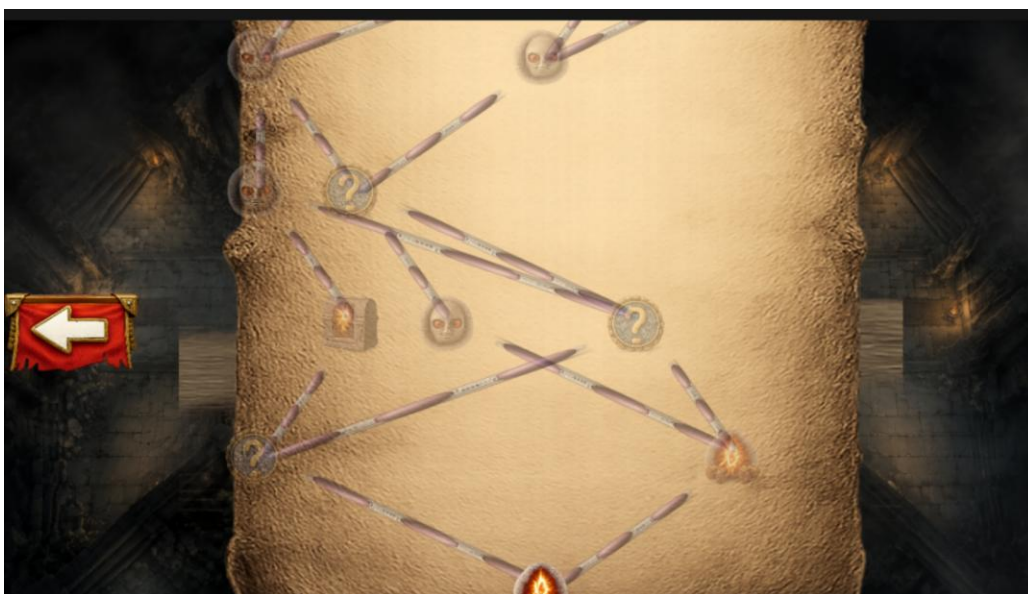


图 7-2 三大关 DAG 跑图界面（RunMapPanel）

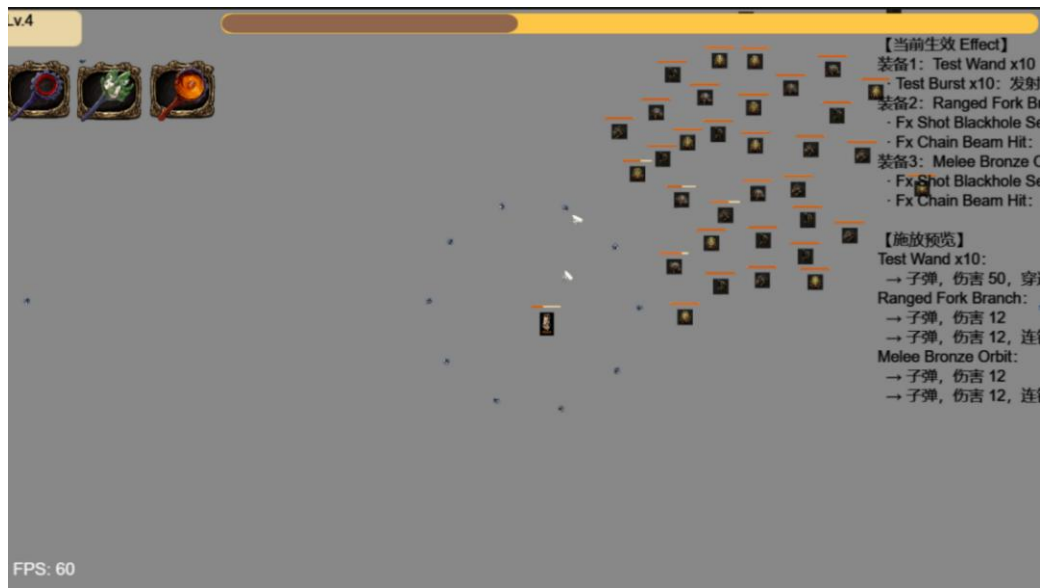


图 7-3 局内战斗场面（同屏多怪与弹幕）



图 7-4 Tab 暂停下的技能/Effect 装配面板

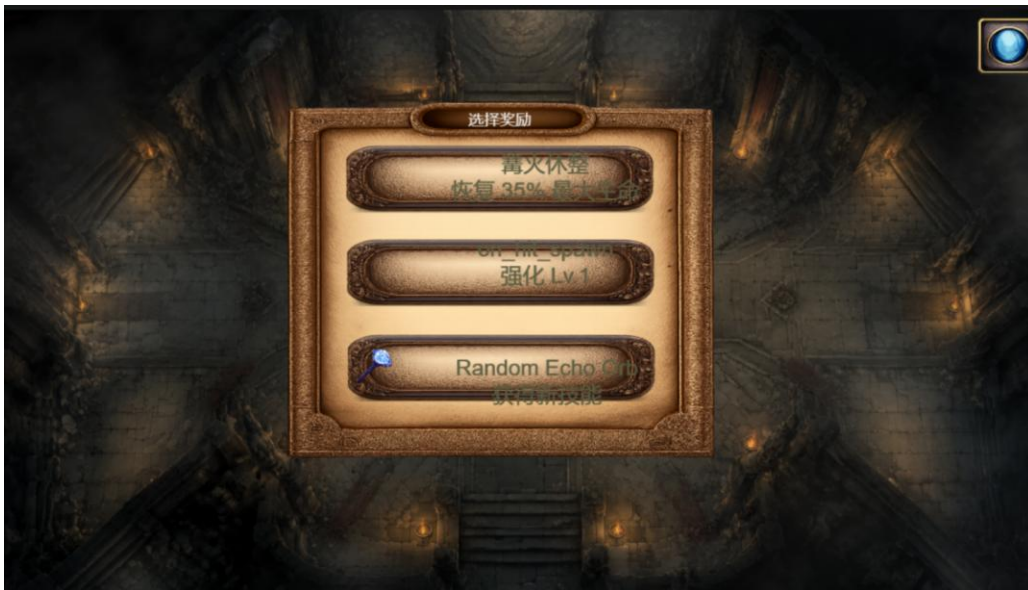


图 7-5 生存胜利后的三选一奖励面板



图 7-6 玩家死亡后的重启面板

7.7 本章小结

本章给出测试策略、单元验证、功能用例、环境与五波性能数据及解读；对象池价值已由第 4-5 波实测支持。图 7-1-7-6 为系统运行截图。

第 8 章 总结与展望

8.1 工作总结

本文围绕 Survivor And Fight，在 LayaAir 3.x 与 TypeScript 上完成 Roguelite 生存战斗的分析、设计、实现与测试，对应关系如下：

(1) 需求与选型（第 1–2 章）：梳理引擎架构、ECS、人群仿真与 H5 性能文献；确定 LayaAir、轻量 ECS、MVC UI、配表战斗的路线。

(2) 总体设计（第 4 章）：给出分层结构、ECS/MVC/Worker 方案，以及时序、可靠性、容错和扩展方式。

(3) 实现（第 5 章）：以 SimpleEcsDemo 为根，实现技能链、子弹、怪物 AI、跑图、Tab 装配和主循环。

(4) 性能与测试（第 6–7 章）：千怪场景池化明显压低 P95；图集抬高重度弹幕波 FPS；用例和脚本验证核心契约。

写作中参考 SyDRA^[18]、Gregory 分层^[7]和 Unity 人群实践^[5]，把模块化、重算分离和 V&V 用到 H5 课设原型上。系统能演示菜单 → 跑图/选关 → 战斗 → 升级装配 → 死亡重启，达到毕设预期。

8.2 不足与展望

(1) 内容与表现：法杖三槽 UI、部分 Effect 的特效和音效尚未补齐。

(2) 架构：同屏实体再多时，ECS 可改为 Archetype/Chunk 存储^[7]；碰撞检测可加空间哈希。

(3) 测试：需要端到端 UI 自动化和更长的定期回归^[18]。

(4) 产品化：局外存档、元进度、帧率与同屏怪物上限等设置仍待做。

(5) 发布：补全美术与适龄说明后，可走静态站点托管。

8.3 社会、健康与文化影响

战斗内容为虚构，怪物和血条偏抽象，避免写实血腥。原型无付费、无诱导分享、不采敏感数据。若上网发布，需遵守网游和未成年人保护规定，标适龄和时长；长时间玩注意眼疲劳，Tab 暂停和跑图休息节点用来调节节奏^[18]。

8.4 绿色节能与兼容性

池化、暂停和 Worker 降低平均 CPU，对续航和发热有帮助^[7]。配表双通道和 Worker 回退提高浏览器兼容性；2D 对 GPU 压力小于大型 3D。后续可在设置里加帧率上限和同屏怪上限，做低功耗档。

8.5 本章小结

Survivor And Fight 说明在 H5 上用轻量 ECS、配表和可关断的性能手段做 Roguelite 原型是可行的。论文叙述与仓库、测试表一致；后续可加强自动化测试和表现层，从答辩 Demo 往可发布小游戏再走一步。

致 谢

论文收尾之际，感谢指导教师在选题、架构和写作上的指点。

感谢计算机学院各位任课教师四年来的培养；感谢同学在项目调试与试玩中提供的帮助；感谢家人在毕业设计期间的理解与支持。

由于本人水平有限，文中疏漏之处恳请各位老师批评指正。

参考文献

- [1] BAABAD A, ZULZALIL H B, HASSAN S, et al. Software architecture degradation in open source software: A systematic literature review[J]. IEEE Access, 2020, 8: 173681-173709.
- [2] GREGORY J. Game engine architecture[M]. 3rd ed. Boca Raton: CRC Press, 2018.
- [3] MUNRO J, BOLDYREFF C, CAPILUPPI A. Architectural studies of games engines -- the quake series[C]//Proceedings of the 2009 International IEEE Consumer Electronics Society's Games Innovations Conference. [S.l.]: IEEE, 2009: 246-255.
- [4] AGRAHARI V, CHIMALAKONDA S. What's inside unreal engine? - A curious gaze![C]//Proceedings of the 14th Innovations in Software Engineering Conference. [S.l.]: ACM, 2021: 1-5.
- [5] GOSLIN M, MINE M R. The Panda3D graphics engine[J]. Computer, 2004, 37(10): 112-114.
- [6] ULLMANN G C, GUÉHÉNEUC Y G, PETRILLO F, et al. SyDRA: An approach to understand game engine architecture[J]. Entertainment Computing, 2025, 52: 100832.
- [7] ULLMANN G C, GUÉHÉNEUC Y G, PETRILLO F, et al. Visualising game engine subsystem coupling patterns[C]//Lecture Notes in Computer Science: Vol. 14455 Entertainment Computing -- ICEC 2023. [S.l.]: Springer, 2023: 263-274.
- [8] Unity Technologies. Unity manual[EB/OL]. (2016)[2026-05-17]. <https://docs.unity3d.com/Manual/index.html>.
- [9] BRIAND L C, BUNSE C, DALY J W. A controlled experiment for evaluating quality guidelines on the maintainability of object-oriented designs[J]. IEEE Transactions on Software Engineering, 2001, 27(6): 513-530.
- [10] REYNOLDS C W. Flocks, herds, and schools: A distributed behavioral model [C]//Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques. [S.l.]: ACM, 1987: 25-34.
- [11] THALMANN D, MUSSE S R. Crowd simulation[M]. London: Springer, 2007.
- [12] CANTRELL W A, PETTY M D, KNIGHT S L, et al. Physics-based modeling of crowd evacuation in the Unity game engine[J]. International Journal of Modeling, Simulation, and Scientific Computing, 2018, 9(4): 1850029.

- [13] BOTT M, MUSSE S R, DE OLIVEIRA L P, et al. Implementing a physics based model of crowd movement using unreal development kit[J]. Journal of Gaming & Virtual Worlds, 2014, 6(3): 275-296.
- [14] HELBING D, FARKAS I, VICSEK T. Simulating dynamical features of escape panic[J]. Nature, 2000, 407: 487-490.
- [15] MCKENZIE F D, PETTY M D, KRUSZEWSKI P A, et al. Integrating crowd-behavior modeling into military simulation using game technology[J]. Simulation & Gaming, 2008, 39(1): 10-38.
- [16] OBERKAMPF W L, ROY C J. Verification and validation in scientific computing[M]. Cambridge, UK: Cambridge University Press, 2010.
- [17] HEIJSTEK W, KÜHNE T, CHAUDRON M R V. Experimental analysis of textual and graphical representations for software architecture design[C]//Proceedings of the 2011 International Symposium on Empirical Software Engineering and Measurement. [S.l.]: IEEE, 2011: 167-176.
- [18] PETTY M D. Verification, validation, and accreditation[M]//Modeling and Simulation Fundamentals. [S.l.]: John Wiley & Sons, 2010: 325-372.

附录 A 名词术语及缩略词

术语/缩略语	含义
ECS	Entity Component System, 实体组件系统
Roguelite	轻 Roguelike, 含元进度与单局随机
DAG	Directed Acyclic Graph, 有向无环图
H5	HTML5 网页游戏
UI2	LayaAir 新一代 UI 组件 (GBox/GButton 等)
Worker	Web Worker, 浏览器后台线程
MoSCoW	Must/Should/Could/Won't 需求优先级方法
V&V	Verification and Validation, 验证与确认
P95	第 95 百分位帧时间, 反映卡顿尖峰

附录 B 核心配表字段示例（节选）

Character 表字段含 id、prefabPath、bodyPrefabPath、hp、maxHp、roleType。玩家和怪物同表，靠 roleType 选预制体和默认值。

Bullet 表：id、prefabPath、duration、speed、damage、penetration。子弹系统按 id 查表生成运动参数与伤害。

Skill 表：id、cooldown、effectSlotCount、图标路径等。一个技能可关联多行 SkillEffect。

SkillEffect 表：id、effect、damage、params、iconPath。effect 取值包括 bullet、modifier_split、modifier_chain、modifier_pierce、direct_damage。

完整 JSON 在 docs/config/。运行前执行 tools/sync-config-to-bin.ps1，把表拷到 bin/config/。

附录 C 需求与测试用例映射（节选）

表 C-1 给出需求标识与测试用例、验证方式的对应关系，便于说明“需求—测试”可追溯性。

表 C-1 需求—测试映射（节选）

需求 ID	需求摘要	验证方式
R-ECS-01	实体销毁时清理全部组件	ecsCorePhase1.verify.ts
R-ECS-02	系统按分组与优先级顺序执行	ecsCorePhase1.verify.ts
R-SKILL-01	三装备栏独立冷却与自动施法	T-F-04，观察多技能轮替
R-SKILL-02	Tab 打开面板暂停战斗	T-F-05，GameSession.paused
R-SKILL-03	拖拽装配写回战斗实体	T-F-06，装配同步用例
R-BULLET-01	穿透/分裂/连锁按配表生效	T-F-04，高密度战斗观察
R-MON-01	波次刷怪与追逐分离	T-F-03，Worker 一致性对比
R-META-01	主菜单进入战斗与计时	T-F-09
R-MAP-01	跑图 DAG Boss 可达	100 次随机种子断言
R-PERF-01	对象池降低帧时间尖峰	表 7-6，第 4 - 5 波
R-PERF-02	动态图集提升千怪 FPS	表 7-4
R-UI-01	升级奖励三选一	T-F-07
R-UI-02	死亡重启面板	T-F-08

附录 D 缺陷记录（项目实测）

表 D-1 缺陷记录表

ID	描述	严重性	状态
B-01	配表未同步至 bin/config 导致技能图标空白	中	已修复（同步脚本）
B-02	Worker 脚本路径错误时回退主线程	低	已修复
B-03	部分 Effect 仅 UI 展示无独立 VFX	低	已知，论文已说明
B-04	法杖三槽运行时与专用 UI 未实现	中	规划项，第 8 章
B-05	千怪首次波次 FPS 低于预热后基线	低	已分析，见第 7 章